PAYLOAD-AWARE TWO-TIER UAV ROUTE PLANNING FOR LAST-MILE DELIVERY

PHASE I REPORT

Submitted By

THIRUMURUGAN RA 3122235001149

MEGHNA MANIMARAN 3122235001079

SHRIKA SP THOTA 3122235001125

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



Department of Computer Science and Engineering

Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University)
Kalavakkam - 603110

NOVEMBER 2026

Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University)

BONAFIDE CERTIFICATE

Certified that this project report titled **"PAYLOAD-AWARE TWO-TIER UAV ROUTE PLANNING FOR LAST-MILE DELIVERY"** is the *bonafide* work of **THIRUMURUGAN RA (3122235001149)**, **MEGHNA MANIMARAN (3122235001079)** and **SHRIKA SP THOTA (3122235001125)** who carried out the project work under my supervision.
Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. T.T. MIRNALINEE
HEAD OF THE DEPARTMENT
Professor
Department of CSE
SSN College of Engineering
Kalavakkam - 603 110

Dr. N. SUJAUDEEN
SUPERVISOR
Assistant Professor
Department of CSE
SSN College of Engineering
Kalavakkam - 603 110

Submitted for project viva-voce examination held on. . . . . .

EXTERNAL EXAMINER **INTERNAL EXAMINER**

ACKNOWLEDGEMENTS

We thank God almighty for giving us the strength and the patience to carry this work through.

We are deeply grateful to our guide, **Dr. N. SUJAUDEEN**, Assistant Professor, Department of Computer Science and Engineering, for his advice, his patience with our many drafts and his steady push to back every claim with a number. This work took its shape in our discussions with him.

Our sincere thanks to **Dr. T.T. MIRNALINEE**, Professor and Head of the Department of Computer Science and Engineering, for her encouragement and support. We thank **Dr. J. SURESH**, Project Coordinator, and the members of our review panel, whose questions at each review made the system better than it would otherwise have been.

We also thank **Dr. S. RADHA**, Principal, for the facilities and support she has provided during our course of study.

We thank the teaching and non-teaching staff of our department who helped us, directly or indirectly, during the project. Finally, we thank our parents and friends for their patience and support throughout.

THIRUMURUGAN RA MEGHNA MANIMARAN SHRIKA SP THOTA

ABSTRACT

Unmanned Aerial Vehicle (UAV) systems for medical and emergency last-mile delivery must balance conflicting objectives: higher flight risk is carried by shorter routes, more battery is drained by low-risk corridors, and the fastest paths are rarely achieved by energy-efficient routing. No single fixed trade-off serves all payload types, yet the same objective weights are applied by existing planners to every mission regardless of the payload. The proposed work is a Payload-Aware Two-Tier UAV Route Planning system, wherein Pareto front construction is separated from route selection and an explicit algebraic bridge is added between Tier 1 routing objectives and Tier 2 payload metadata. In Tier 1, the NAMOA* multi-objective search is executed over a three-dimensional airspace lattice built from terrain elevation, OpenStreetMap buildings and restricted zones, with wind-dependent, direction-aware edge costs, producing a Pareto-optimal route set across distance, energy consumption, and flight risk. In Tier 2, four operator-assigned payload tags are read and objective weights derived: w_{risk} is scaled by fragility class, w_{distance} by delivery urgency, and w_{energy} by payload weight class, while mission priority amplifies the dominant weight. Every Pareto point is scored by these weights via a utility function, and the argmax is selected as the mission route. The principal contribution is a formula-driven mapping from metadata to routing weights, linking mission context directly to Pareto selection without expert intervention. Route choice is set by payload properties rather than context-blind diversity criteria. The Pareto front is stored and rebuilt only when the graph changes, its validity window expires or the weather drifts, serving all missions between updates.

TABLE OF CONTENTS

ABSTRACT iii

LIST OF TABLES viii

LIST OF FIGURES ix

LIST OF ABBREVIATIONS x

1 INTRODUCTION 1

1.1 MOTIVATION 2

1.2 BACKGROUND 3

1.2.1 Building a front versus choosing from it 3

1.2.2 Evidence from real deployments 4

1.2.3 Regulation 5

1.3 PROBLEM DEFINITION 5

1.3.1 Formal statement 6

1.3.2 Weather dependence 7

1.3.3 Inputs, outputs and success criteria 7

1.3.4 Scope 7

1.4 ORGANIZATION OF THE REPORT 8

2 LITERATURE SURVEY 9

2.1 PAPERS RELATED TO THE PROBLEM IDENTIFIED 9

2.1.1 Pareto-optimal particle swarm optimisation with entropy selection 9

2.1.2 Crowding-distance NSGA-II for urban UAV path planning 10

2.1.3 Multi-strategy improved NSGA-II for 3-D trajectory planning . . . 11
2.1.4 NSGA-II and MOPSO . . . 12
2.1.5 Exact multi-objective A* search . . . 12
2.1.6 Payload-mass-aware trajectory planning . . . 13
2.1.7 Quality indicators for Pareto fronts . . . 13
2.1.8 Effect of drone transport on medical payloads . . . 14
2.1.9 Summary of the reviewed work . . . 14
2.2 GAPS IDENTIFIED . . . 16
2.3 RESEARCH OBJECTIVES . . . 17

3 PROPOSED METHODOLOGY 19
3.1 ARCHITECTURE DESIGN . . . 20
3.2 AIRSPACE GRAPH BUILDER (MODULE 1) . . . 21
3.2.1 Environment data acquisition . . . 22
3.2.2 Lattice generation and node selection . . . 22
3.2.3 Edge construction and pruning . . . 23
3.2.4 Static risk terms . . . 24
3.2.5 Wind-aware energy model . . . 25
3.2.6 Weather-dependent risk and the go/no-go gate . . . 26
3.3 TIER 1 PARETO FRONT ENGINE (MODULE 2) . . . 28
3.3.1 Labels and dominance . . . 28
3.3.2 Admissible vector heuristic and pruning . . . 29
3.3.3 Truncation and front thinning . . . 29
3.3.4 Why an exact method . . . 31
3.3.5 Pareto front store and refresh . . . 31
3.4 BASELINE PLANNERS (MODULE 3) . . . 32
3.5 TIER 2 PAYLOAD METADATA INTERPRETER (MODULE 4) . . . 32

3.5.1 Semantic mapping 33
3.5.2 Weight derivation 33
3.5.3 Utility scoring and selection 34
3.5.4 Reference presets 35
3.5.5 Battery feasibility 36
3.5.6 Weather hold and decision record 36
3.6 EVALUATION AND BENCHMARKING (MODULE 5) 38
3.6.1 Front quality measures 38
3.6.2 Tier 2 evidence 39
3.6.3 Tag sweep and monotonicity 39
3.6.4 The evaluation cycle 39
3.7 MISSION EXPORT AND DASHBOARD (MODULE 6) 40
3.7.1 Mission file generation 40
3.7.2 Operator dashboard 40
3.8 SIMULATION AND SITL VALIDATION (MODULE 7) 41
3.8.1 Bundle, clearance and frame contract 41
3.8.2 Mission sequence 42

4 EXPERIMENTAL RESULTS AND PERFORMANCE ANALYSIS 43
4.1 DATASET DESCRIPTION 43
4.1.1 Synthetic airspace graphs 43
4.1.2 Real-region lattice 44
4.1.3 Payload tag space and simulation bundle 45
4.2 ECOSYSTEM 46
4.2.1 Hardware configuration 46
4.2.2 Software environment 46
4.3 EXPERIMENTS CONDUCTED 47
4.4 PERFORMANCE ANALYSIS 48

4.4.1 Tier 1 front quality against the baselines 48
4.4.2 Effect of wind speed and direction 50
4.4.3 Directional edge costs 52
4.4.4 Contribution of each risk layer 52
4.4.5 Tier 2 selection 53
4.4.6 Simulation results 55
4.4.7 Comparison with existing work 56

5 SOCIAL IMPACT AND SUSTAINABILITY 57
5.1 SOCIAL IMPACT 57
5.2 SUSTAINABILITY 58

6 CONCLUSION AND FUTURE WORK 59
6.1 CONCLUSION 59
6.2 FUTURE WORK 60

REFERENCES 61

A CONFIGURATION PARAMETERS 63

B AUTOMATED TEST SUITES 64

C CODE ORGANISATION 66

LIST OF TABLES

1.1 Inputs, outputs, constraints and success criteria 8

2.1 Summary of the reviewed work 14
2.2 Gaps identified and the research objectives that address them . . . 18

3.1 Risk components (every $\rho_k \in [0, 1]$) and their weights 24
3.2 Semantic mapping from payload tags to Tier 1 objectives 33
3.3 Reference payload presets and their derived weights 35

4.1 Graphs and scenarios used in the experiments 44
4.2 Environment data sources for the real-region lattice 45
4.3 Experiments carried out in Phase I so far 47
4.4 First-pass benchmark on the synthetic scenarios (HV higher is better; IGD and Δ lower are better) 49
4.5 Effect of wind direction on the S2 front (25 km/h wind, 35 km/h gust) 50
4.6 Directional edge costs on a flat lattice, 25 km/h wind from 90° . . 52
4.7 Ablation on the 6 km lattice (fixture layers, 30 km/h wind from 225°) 52
4.8 Tier 2 picks on the S2 (16 routes) and S3 (115 routes) fronts . . . 53
4.9 Comparison with the reviewed planners 56

A.1 Main airspace graph and cost model parameters 63
A.2 Front store and Tier 2 parameters 64

B.1 Test suites and what they verify 64

C.1 Source files and the modules they implement 66

LIST OF FIGURES

3.1 Methodology overview: Tier 1 Pareto front construction feeding Tier 2 payload-aware route selection, with mission output, UAV control and evaluation 19
3.2 System architecture. Tier 1 builds and stores the front per region and weather window; Tier 2 selects a route per payload tag; Modules 6 and 7 export and fly it 20
3.3 Gazebo and ArduPilot SITL simulation pipeline 41
3.4 Mission sequence from payload tag entry to the flown route 42

4.1 Hypervolume per method and scenario (first pass) 49
4.2 Front size, minimum energy and minimum risk on S2 as a 25 km/h wind turns through 360°; the dashed line is calm air 51
4.3 Tier 2 routes for the four presets on the 2 km lattice 54
4.4 Gazebo Harmonic and ArduPilot SITL flying an uploaded Tier 2 mission in AUTO: MAVProxy console (left), QGroundControl mission view (right) 55

LIST OF ABBREVIATIONS

AGL	Above Ground Level
AMSL	Above Mean Sea Level
DEM	Digital Elevation Model
GPS	Global Positioning System
HV	Hypervolume
IGD	Inverted Generational Distance
MAVLink	Micro Air Vehicle Link
MOPSO	Multi-Objective Particle Swarm Optimisation
NAMOA*	New Approach to Multi-Objective A*
NFZ	No-Fly Zone
NSGA-II	Non-dominated Sorting Genetic Algorithm II
OSM	OpenStreetMap
POPSO	Pareto-Optimal Particle Swarm Optimisation
QGC	QGroundControl
RMS	Root Mean Square
SDF	Simulation Description Format
SITL	Software In The Loop
SORA	Specific Operations Risk Assessment
UAV	Unmanned Aerial Vehicle
WMO	World Meteorological Organization

CHAPTER 1

INTRODUCTION

Unmanned aerial vehicles (UAVs) are a practical option for last-mile delivery where ground transport is slow or impossible: medical supplies for remote clinics, emergency drug drops after a disaster, and diagnostic samples that should not spend an hour shaking in the back of a van. Every one of these flights goes from a dispatch hub to a delivery point, and the air in between holds buildings, hills, restricted zones and wind. Three things compete when we choose the route: the distance flown, the battery spent, and the share of the trip spent in risky air. Improve one and another usually gets worse.

Multi-objective path planning was designed for exactly this. Such a planner carries distance, energy and risk separately through the search and hands back a Pareto-optimal front, a set of routes where no route can gain on one objective without losing on another [7]. The front lists the trade-offs on offer. Which one to fly is left open.

Most systems take a shortcut here. They merge the three objectives into one weighted sum, give the scalar cost to a plain planner such as A*, and let the operator set the weights before each flight. If every mission is the same kind, that is fine. Change the payload and it breaks: emergency medicine wants a very different weighting from a fragile blood sample, and both differ from a box of relief supplies [4]. The process of re-tuning and re-running the search algorithm between flights consumes time, and in the event of an emergency, nobody with the experience of the planner is available.

Our Payload-Aware Two-Tier UAV Route Planning system takes that step away. Tier 1 builds the Pareto front for a delivery region once by using an exact multi objective graph search over 3D airspace lattice. This lattice is used because its edge costs depend on terrain, buildings, restricted zones and the way the wind blows. We keep that front until the graph changes, the front expires or the weather drifts. Tier 2 runs for each mission. The operator enters four payload tags, a fixed formula turns them into objective weights, every stored route is scored, and the top one comes back in well under a millisecond. This is then exported as an ArduPilot mission and flown in a Gazebo and ArduPilot software-in-the-loop (SITL) simulation.

1.1 MOTIVATION

Three delivery scenarios that come up again and again in medical and emergency drone work motivate this project, and their routing needs do not agree with each other.

Fragile medical samples such as tissue biopsies and blood specimens must arrive intact, and an operator will want them kept out of rough air when a smoother corridor exists. The priority is low flight risk: little time in turbulent or crosswind-exposed corridors and few sudden altitude changes. We would take a longer route if in case the air is calmer.

Emergency medicine such as insulin or antivenom has a strict time window. The priority is the shortest route, so that transit time is as low as possible, and risk and energy come second. If the delivery window is fifteen minutes, the shortest path wins.

Disaster-relief supplies such as water purification tablets and ration packs are heavier than medical payloads, and they are neither fragile nor urgent. Here the priority is low battery drain, so that one charge covers as many drops as possible.

A real delivery service sees all three, sometimes within the same hour. A planner with fixed weights treats them identically, and one that lets the operator change the weights pushes an optimisation task onto someone dispatching a drone in a hurry. We could not find a UAV delivery planner that switches between these routing strategies from a description of the payload, without an expert in the loop and without running the search again. That is what motivated the two-tier design.

A second motivation is wind. None of the planners we reviewed resolve wind direction per edge, yet for a quadcopter cruising at 36 km/h a 25 km/h wind is not a detail: flying into it costs far more energy than flying with it, and crosswind makes the ride rough. Section 4.4.2 shows that at a fixed wind speed, the direction alone changes the energy of the cheapest available route by 27.5%. A planner blind to direction gets this wrong on any windy day, and so does any payload-aware choice built on top of it.

1.2 BACKGROUND

1.2.1 Building a front versus choosing from it

A routing front can be built in two ways. Exact Graph Search algorithms like NAMOA* [11] keep all non-dominated paths at each node. Conversely, other algorithms like NSGA-II [6] and MOPSO [5] perform better on large search

spaces but use random procedures. Typical route selection strategies involve the use of crowding distance technique [6, 18], an entropy criterion coupled with a weighted sum [4], or even manual intervention. None of these techniques consider the payload being delivered by the drone. In the energy model [21], payload is expressed through mass which impacts edge cost without affecting the relative trade-offs. Our investigation suggests that no algorithm has considered the payload metadata for selecting a particular point on the Pareto front.

1.2.2 Evidence from real deployments

Usage of services highlights the instances in which payload is significant, but the available information is scant. Swiss Post used the Matternet drones for transporting laboratory samples from one hospital to another and then discontinued its service after two incidents occurred in 2019. First incident was in January when the drone faced an electrical fault, which resulted in an interruption of power supply to the GPS system, and the drone had to land in Lake Zurich urgently. Second incident happened in May when parachute of the drone got entangled in its frame, resulting in a descent into trees near some children playing in kindergarten [2]. Both were equipment failures, so they say nothing about route choice. They do show that laboratory samples are routinely flown over water and populated ground, where the choice of corridor has consequences.

Laboratory studies of medicines and blood samples flown by drone found no damage on ordinary flights [14, 19, 20], but none of them varied the route (Section 2.1), so in this work the fragility tag is a precaution the operator sets, not a measured degradation effect.

Zipline's blood-delivery network in Rwanda serves hospitals up to about 80 km from its distribution centre, and each aircraft follows a predetermined flight plan [9]. Its engineers treat payload weight as a limit on range. That is the energy side of the problem Tier 2 addresses: a heavier load shortens how far the drone can go, so the best route depends on what is on board.

1.2.3 Regulation

In India, small civil drones fly under the Drone Rules, 2021 [13], which treat 120 m above ground as the ceiling for flight without special permission. Ground risk is commonly ranked with the SORA framework [10]. Our airspace model uses both.

1.3 PROBLEM DEFINITION

Existing UAV route planners either apply a fixed geometric selection rule over the Pareto front that ignores the payload, or fold the objectives into one hand-tuned weighted sum before the search begins, which fixes the trade-off regardless of what is being carried. When the payload changes, an expert has to re-tune the weights and run the search again, because no explicit mapping exists from a payload description to the trade-off it implies. In the field, that expert is not available.

1.3.1 Formal statement

We are given:

1. an airspace graph $G = (V, E)$ in which every edge carries a three-objective cost vector $\mathbf{c}(e) = [d(e), \varepsilon(e), \rho(e)]$ for flight distance, estimated energy and flight risk;

2. a Pareto front $\mathcal{P}$ of non-dominated routes computed in advance over G; and

3. a payload metadata tuple $(F, W, U, P_{\text{lvl}})$ holding fragility class $F \in \{1, \ldots, 5\}$, payload weight class $W \in \{1, 2, 3\}$, delivery urgency $U \in \{1, 2, 3\}$ and priority level $P_{\text{lvl}} \in \{1, \ldots, 5\}$.

The problem is to select the route $p^* \in \mathcal{P}$ that best meets the requirements implied by the payload metadata, without expert re-tuning and without running the multi-objective search again.

The inputs are values an operator can assign from domain knowledge ("this vial is fragile, $F = 5$; the delivery is not urgent, $U = 1$"). Selection must be deterministic and reproducible, and it must finish within one second of the tags being entered. Three further conditions follow from the gaps identified in Chapter 2. The selected route must depend on $(F, W, U, P_{\text{lvl}})$ and not on the geometry of the front alone. The front needs to be reusable during multiple missions before the next refresh instead of constructing it from scratch each time. Lastly, the conversion from $(F, W, U, P_{\text{lvl}})$ to p^* needs to be stated in terms of a mathematical formula; anything else is unacceptable.

1.3.2 Weather dependence

Weather affects the cost of edges. Let ω be a measurement of wind speed and direction, gusts, precipitation, and visibility. An edge then costs $\mathbf{c}(e;\omega) = [d(e), \varepsilon(e;\omega), \rho(e;\omega)]$ and the front becomes $\mathcal{P}(\omega)$. The distance will always be constant, while energy and risk depend on the intersection angle of the wind with each edge. Selection is done as before. However, the difficulty is that an old front can be used only for as long as ω does not deviate too much from the measurements it was built for; therefore, the front must be updated by time or when the weather deviates. If ω is outside the go/no-go limits, then there is no selection and the mission is rescheduled.

1.3.3 Inputs, outputs and success criteria

Table 1.1 specifies the inputs, the outputs, and the tests each of them needs to pass.

1.3.4 Scope

This report covers Phase I which delivers the complete software system and its simulation. In Phase II the same system is transferred onto a Pixhawk-equipped F450 quadcopter with a Raspberry Pi 5 companion computer and is tested in outdoor flights. A Tier 1 front is trusted only for a bounded window. Once the window runs out, or the weather moves past set thresholds, it is rebuilt; until then Tier 2 alone serves every mission. Multi-UAV coordination, dodging moving obstacles and real-time replanning are outside our scope.

TABLE 1.1: Inputs, outputs, constraints and success criteria

Item	Definition
Region input	Centre point and box size; terrain (DEM), OpenStreetMap buildings, restricted zones, water and land use for that box
Weather input	Snapshot ω from Open-Meteo: wind at 80 m, gust, rain, visibility, WMO weather code
Mission input	Depot, destination and payload tuple $(F, W, U, P_{\text{lvl}})$
Tier 1 output	Stored Pareto front $\mathcal{P}(\omega)$ with graph hash, weather snapshot and expiry time
Tier 2 output	Selected route p^* with waypoints (latitude, longitude, altitude), weights, ranks and point type, or a HOLD advisory
Mission output	QGC WPL 110 `.waypoints` and QGroundControl `.plan` files
Hard constraints	Cruise at least 60 m above ground and 15 m above roofs; at most 120 m above ground; no entry into restricted zones; route energy within 80% of battery capacity
Tier 1 criteria	Hypervolume and IGD at least as good as every baseline on each scenario; no dominated or duplicate routes in the front
Tier 2 criteria	Selection under 1 s; zero monotonicity violations over all 225 tag combinations; non-extreme picks for mixed payloads
Simulation criteria	Cross-track error, clearance margin and landing offset of the flown route in SITL and Gazebo

1.4 ORGANIZATION OF THE REPORT

There are six chapters, then the references and three appendices. Chapter 1 gives the motivation and the problem. Chapter 2 reviews various near papers, what they leave out, and the objectives that we took from that. Chapter 3 explains the architecture and its seven modules including the algorithms inside them. Chapter 4 covers datasets, hardware and software, the experiments run so far and their results. Chapter 5 looks at social impact and sustainability, and Chapter 6 looks at the plan for the rest of Phase I and for Phase II.

CHAPTER 2

LITERATURE SURVEY

Work on multi-objective UAV path planning comes from both graph search and evolutionary optimisation [7]. The main thing we asked was that once a planner holds a set of good routes, how does it choose the one to fly, and does the payload get a say? Below we show the papers which are nearest to that question and the few studies of what drone transport does to medical payloads. The chapter ends with gaps we found and the objectives we decided from them.

2.1 PAPERS RELATED TO THE PROBLEM IDENTIFIED

2.1.1 Pareto-optimal particle swarm optimisation with entropy selection

Chokkalingam and Ganesan [4] propose a Pareto-optimal particle swarm optimiser (POPSO) for UAV paths in changing surroundings, which has four parts. Grid-based and random particles are mixed at the start which ensures that the swarm begins with varied paths. Path length, energy, an inverse safety term, a penalty for moving obstacles and a weather factor are the factors that determine the cost. Inertia weight and learning coefficients shift during the run hence pushing exploration when progress stalls and exploitation when it does not. Last, a hybrid Pareto-entropy step orders the archive by Pareto rank and by an entropy

value built from inter-solution distances, which keeps the archive varied. On real terrain images with added obstacles it produced well-distributed fronts and better path efficiency and safety than the methods it was compared with.

Two details matter for our work. The weather factor is $W = \gamma(1 + v_{\text{wind}}/v_{\max})$, which depends on wind speed only, so a path flown into the wind and the same path flown with it are costed alike. And the final route is not chosen by the entropy mechanism. For that the paper uses a separate weighted sum of the normalised objectives, with weights "typically determined empirically based on mission priorities". The same sum drives particle updates and picks the one solution that gets deployed. So the last step is still a hand-set weighted sum, and the payload does not enter it.

2.1.2 Crowding-distance NSGA-II for urban UAV path planning

Ren et al. [18] plan UAV paths in an urban environment with two objectives, distance and safety. They store the city as an octree, which splits 3-D space level by level and uses little memory over large empty areas, and add a safety index map rating each cell by its nearness to obstacles. A* paths seed the population and NSGA-II evolves it. They show one case where the usual crowding distance favours an odd point, and redefine crowding as the mean distance to the k nearest solutions. The resulting CDNSGA-II quickly finds well-spread, collision-free Pareto paths.

A weighted sum, they argue, needs its weights fixed beforehand, which is hard, so they build the front first and choose from it later. The pick, though, is made by

crowding distance, which measures how isolated a point is on the front and has no notion of urgency, fragility or payload weight.

2.1.3 Multi-strategy improved NSGA-II for 3-D trajectory planning

Deng et al. [7] go after two familiar faults of evolutionary planners in 3-D. They get trapped in local optima, and fast convergence tends to cost population diversity. Their MS-INSGA2 framework seeds the population with linearly interpolated trajectories that pass a safety check, couples a dynamic guidance step with crossover and mutation rates that adapt to the state of the population, and adds a local search that refines the Pareto front and smooths the trajectories. Three objectives are optimised: path length, collision risk from terrain and cylindrical obstacles, and smoothness measured as the change in curvature between segments. On the ZDT, DTLZ and WFG benchmark suites, with a Wilcoxon signed-rank test, the framework outperformed its baselines, and on two 3-D planning tasks it beat eleven other algorithms, lowering the average fitness by 27.87% and 32.06% and the best fitness by 6.91% and 14.18%.

The paper is about search quality and trajectory smoothness. Payload and delivery context do not appear among the objectives, and wind is listed as future work rather than modelled. As with the other evolutionary planners, the front is rebuilt for each planning task.

2.1.4 NSGA-II and MOPSO

NSGA-II [6] is still the reference evolutionary algorithm for several objectives. It brought fast non-dominated sorting, elitism by merging parents and offspring, and the crowding distance, which prefers solutions in sparse regions of the front. MOPSO [5] extended particle swarm optimisation with an external archive of non-dominated solutions, leader selection on an adaptive grid and a mutation operator. Both are fast, stochastic and widely used as baselines. Our MOPSO baseline adapts the second to graph routes (Section 3.4), and the crowding distance is one of the competing selection rules that Tier 2 is compared against (Section 3.6).

2.1.5 Exact multi-objective A* search

Mandow and Pérez de la Cruz [11] analyse NAMOA*, a multi-objective extension of A*. Instead of one cost per node, NAMOA* keeps a set of non-dominated cost vectors (labels) per node and expands labels, not nodes. With an admissible heuristic it returns the complete set of Pareto-optimal solution costs. The paper proves that, when the heuristic is consistent, NAMOA* expands no more labels than any other admissible multi-objective search algorithm given the same heuristic, and that a better-informed heuristic never makes it expand more. NAMOA* is the starting point of our Tier 1 engine. Its exactness is the reason we chose it over a population method: the front is the same on every run, and interior points that a payload might need are never lost to chance.

2.1.6 Payload-mass-aware trajectory planning

Tsoutsouras et al. [21] look at drones shared by several users, where packages of different mass are dropped at different waypoints before their deadlines. They build a power model in which propulsion power depends on both payload mass and flight speed, and formulate trajectory planning as minimising deadline misses under energy, waypoint and mass constraints, a problem close to the travelling salesman problem. A greedy heuristic produces a first trajectory and an optimiser improves it, and the whole computation runs on the drone, in under about 320 ms on an ARM Cortex-A9.

This is the closest work to ours in that payload matters to the route. The way it matters is different, though. Mass is a continuous physical parameter inside the energy and speed model, and the planner returns one trajectory. There is no Pareto front, and no discrete payload description such as fragility or urgency that chooses between trade-offs.

2.1.7 Quality indicators for Pareto fronts

Zitzler and Thiele [22] introduced the size of the dominated space, now called the hypervolume (HV): the volume of objective space a front dominates up to a reference point. It rewards both closeness and spread, and it is the main front-quality measure in our benchmarks.

2.1.8 Effect of drone transport on medical payloads

Three studies measure what drone flight does to medical cargo. Oakey et al. [14] recorded vibration on fixed-wing and multi-copter drones and found levels up to three times those of road transport, at higher frequencies, but insulin carried on those flights showed no aggregation. Theobald et al. [20] tried flying monoclonal antibody infusions on sixty flights with vibration between 1.5 and 3 g and found zero change in particle size or aggregation. Stierlin et al. [19] compared drone and car transport of blood samples and found them comparable, with small differences in five analytes.

So the payloads that were tested come unharmed. The studies do not say what happens to a more fragile cargo, or whether a calmer corridor would help, because none of them changed the route. We read this as a reason to treat fragility as an operator's precaution and not as a proven damage model.

2.1.9 Summary of the reviewed work

The papers are shown in Table 2.1.

TABLE 2.1: Summary of the reviewed work

No.	**Author / Year**	**Method**	**Findings**	**Limitation for our problem**
1	Chokkalingam and Ganesan (2026) [4]	POPSO with hybrid initialisation, adaptive swarm parameters and Pareto-entropy archive	Well-distributed fronts; better path efficiency, safety and adaptability than compared methods	Final route chosen by a weighted sum with empirically set weights; wind enters as a speed-only factor; payload absent

No.	Author / Year	Method	Findings	Limitation for our problem
2	Ren et al. (2019) [18]	CDNSGA-II over an octree city model with a safety index map; A* seeding	Spread-out, collision-free paths for distance and safety	Selection by crowding distance only; no mission context; no wind
3	Deng et al. (2026) [7]	MS-INSGA2 with safe interpolated seeding, adaptive operators and local search	Beats eleven algorithms on 3-D tasks; better benchmark indicators	No payload or delivery context; wind left to future work; front rebuilt per task
4	Deb et al. (2002) [6]	NSGA-II: fast non-dominated sorting, elitism, crowding distance	Better spread and convergence than PAES and SPEA on test problems	Crowding distance is purely geometric
5	Coello Coello et al. (2004) [5]	MOPSO with external repository, adaptive grid and mutation	Competitive fronts at low cost	Stochastic; may miss interior points of the front
6	Mandow and Pérez de la Cruz (2010) [11]	NAMOA*: label-based multi-objective A*	Exact front with admissible heuristics; efficiency results under consistent heuristics	Says nothing about which route to fly
7	Tsoutsouras et al. (2020) [21]	Mass- and speed-aware power model; heuristic plus optimiser for deadline-constrained multi-drop trajectories	Near-optimal trajectories on board in about 320 ms	One trajectory, no front; mass is a physical parameter, not a selector
8	Zitzler and Thiele (1999) [22]	Comparative study of MOEAs; SPEA; dominated-space indicator	Hypervolume as a front-quality measure	Evaluation tool only
9	Oakey et al. (2021); Theobald et al. (2023); Stierlin et al. (2024) [14, 19, 20]	Vibration and sample-quality measurements on drone flights	Tested medicines and blood samples unaffected by ordinary flights	Route never varied; no evidence for or against route-based protection

2.2 GAPS IDENTIFIED

Front diversity acts as the main criteria of design for two planners analyzed, but only one of them uses it in choosing a route: Ren et al. [18] choose a route using a point selected directly from the front on the basis of crowding distance. Chokkalingam and Ganesan [4] use entropy only to keep the archive diverse during optimisation, and fall back on a weighted sum, with weights set per mission, to make the final pick. Deng et al. [7] improve the front itself and leave the choice open. Tsoutsouras et al. [21] do not build a front at all: they put continuous payload mass into the energy model and adapt speed as mass changes, but never choose between competing trade-offs by payload.

So none of these works lets a payload property act as an input to a Pareto-point selection step. This follows from how the mechanisms are defined: entropy and crowding distance are functions of the front's geometry, and mass-aware adaptation is a function of the physical model. Adding payload information would mean replacing the selection rule outright, which is what our Tier 2 does.

We state the gaps as follows.

Gap 1. No payload-aware route selection. Existing UAV multi-objective planners apply a fixed geometric criterion that ignores the payload, a hand-tuned weighted sum, or no selection mechanism at all.

Gap 2. No reuse of a front between mission types. Every planner we read reruns the whole search when the mission changes. Nothing separates building the front, which depends on the region and the weather, from choosing a route, which depends on the payload.

Gap 3. No algebraic bridge from payload to trade-off. We found no written, repeatable rule from a payload description to the trade-off it calls for (fragility to low risk, urgency to short distance, weight to low energy).

Gap 4. Wind direction is not resolved per segment. When weather is modelled at all, it is one speed-only factor applied everywhere [4]. A headwind and a tailwind on the same leg cost the same, and a stored front carries no record of the weather it was built for, so there is no rule for when it goes stale.

2.3 RESEARCH OBJECTIVES

Each objective below answers one or more of these gaps.

1. To cast UAV route planning as precomputing a Pareto front over a delivery airspace graph, and to justify exact search over stochastic methods.

2. To build a 3-D airspace lattice from terrain, buildings, restricted zones, water and land use, with wind- and heading-dependent energy and risk on each edge, and a rule for when a stored front must be rebuilt.

3. To define a payload metadata schema and a written formula turning mission context (fragility class, payload weight class, delivery urgency, priority) into weights for picking a Pareto point, with no expert tuning.

4. To score the Tier 1 front against single-objective A* and MOPSO by hypervolume, inverted generational distance and spread.

5. To gather evidence that Tier 2 picks routes that match the payload, including non-extreme routes for mixed payloads, and does so within one second.

6. To export the selected route as an autopilot mission and verify it in a Gazebo and ArduPilot SITL simulation before hardware flights in Phase II.

Table 2.2 links each gap to its objectives.

TABLE 2.2: Gaps identified and the research objectives that address them

Gap	**Objectives**	**How the system addresses it**
Gap 1: no payload-aware selection	3, 5	Tier 2 derives weights from four payload tags and scores every route on the front; argmax gives the mission route
Gap 2: no front reuse	1, 4	Tier 1 builds an exact NAMOA* front once per region and weather window; a front store reuses it for every mission until it expires or the weather drifts
Gap 3: no algebraic bridge	3, 5	Weight formula $w_{\text{risk}} \propto 2F$, $w_{\text{dist}} \propto 3U$, $w_{\text{energy}} \propto 2W$, with priority amplification; deterministic and checked for monotonicity over all 225 tag combinations
Gap 4: wind direction not resolved	2, 6	Headwind and crosswind computed per edge from its bearing; gust, turbulence and a go/no-go gate; front stored with its weather snapshot and refreshed on drift

CHAPTER 3

PROPOSED METHODOLOGY

The whole system rests on one design decision: Pareto front construction (Tier 1) is kept apart from route selection (Tier 2). The front depends on the region and the weather, so it is built once per delivery region and refreshed when it goes stale. The choice of route depends on the payload, so it is made per mission, against whatever front is current. A mission that arrives between refreshes runs only Tier 2, which reads the payload tags and returns a route in a fraction of a millisecond.

Figure 3.1 shows the flow of the process from the environment data to the implementation of the mission. Baseline planners, as well as an evaluation module, score each step in the process.

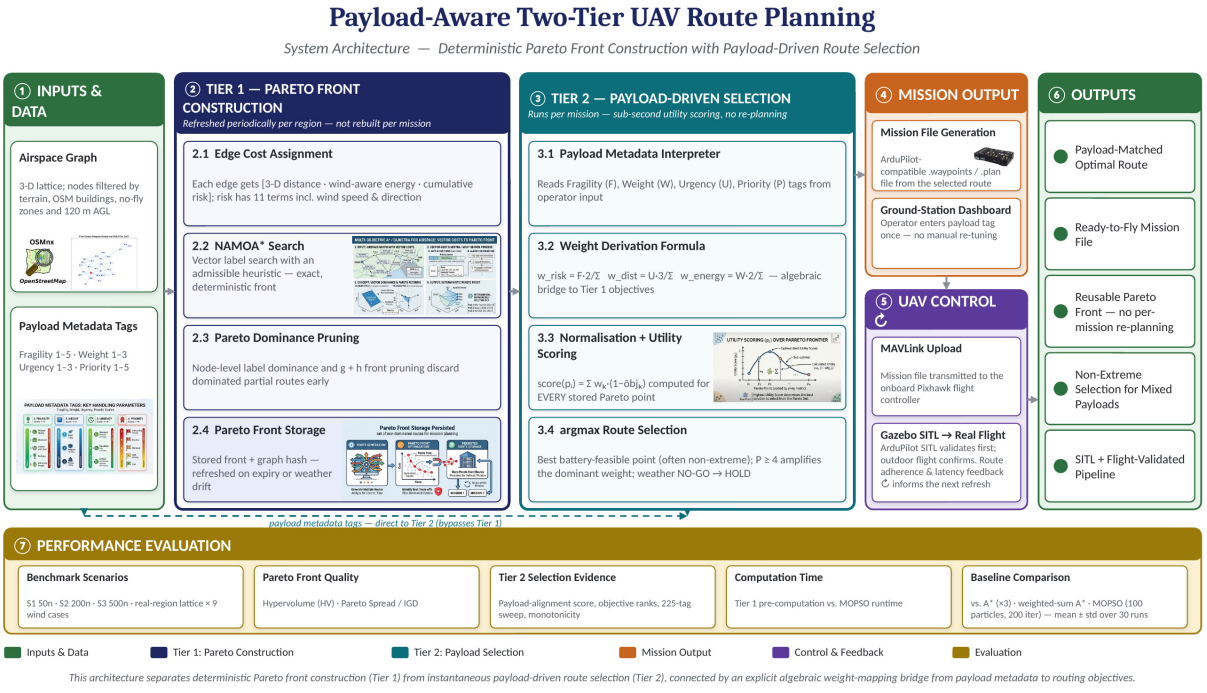


FIGURE 3.1: Methodology overview: Tier 1 Pareto front construction feeding Tier 2 payload-aware route selection, with mission output, UAV control and evaluation

3.1 ARCHITECTURE DESIGN

Figure 3.2 shows the system architecture, which has five layers. The environment data layer fetches weather, terrain and OpenStreetMap layers and caches them on disk, so that a network failure falls back to the last good copy instead of stopping the planner. Module 1 transforms these layers into a directed graph of the airspace where the edges are linked with a cost vector having three objectives. Tier 1 (Module 2) computes the Pareto front over that graph and keeps it in a front store, together with what is needed to decide when it is stale. Tier 2 (Module 4) reads the operator's payload tags and returns one route from the stored front, or a HOLD advisory when the weather or the battery rules the mission out. The output layer (Modules 6 and 7) writes the mission file and flies it in Gazebo with ArduPilot SITL. Modules 3 and 5 sit beside the main path. They run the baseline planners and compute the quality measures used in Chapter 4.

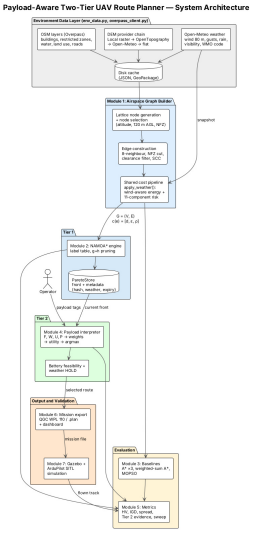


FIGURE 3.2: System architecture. Tier 1 builds and stores the front per region and weather window; Tier 2 selects a route per payload tag; Modules 6 and 7 export and fly it

Every edge $e = (u, v)$ of the airspace graph carries the cost vector

$$\mathbf{c}(e;\omega) = \big[d(e),\ \varepsilon(e;\omega),\ \rho(e;\omega)\big] \tag{3.1}$$

where $d(e)$ is the three-dimensional length of the segment in kilometres, $\varepsilon(e;\omega)$ is the wind- and grade-aware energy proxy and $\rho(e;\omega)$ is the cumulative risk exposure along the segment under weather snapshot ω. Section 3.2 defines all three. The three objectives are to minimise distance, energy and risk. Route p dominates route q, written $p \prec q$, when p is no worse than q on all three objectives and strictly better on at least one. The Pareto front $\mathcal{P}$ is the set of routes that no other route dominates.

There are seven modules in the code. Their numbers match the source files, which is why the baselines are Module 3 and Tier 2 is Module 4. For every module, the following sections state its objective, inputs, processing, and outputs.

3.2 AIRSPACE GRAPH BUILDER (MODULE 1)

Module 1 builds the delivery airspace graph $G = (V, E)$ based on three parameters: the region itself (its center point, size of the box, and lattice spacing), environmental layers to be considered later, and a weather sample ω. The result is a directed graph using the NetworkX library [8]. All nodes have their coordinate data and height information, and all edges have their static features, component risk vector, and cost vector $[d, \varepsilon, \rho]$. The weather go/no-go state is also stored on the graph for Tier 2. Two builders share one cost pipeline: one lays a lattice over a real region (below); the other makes seeded synthetic graphs for controlled

benchmarks (Section 4.1). Algorithm 1 gives an overview of the lattice constructor.

3.2.1 Environment data acquisition

Weather information comes from the Open-Meteo forecasting service [15], which provides 10 m wind, gusts, precipitation, visibility, and the WMO code, as well as hourly wind speed and direction at 80 m. If there are any connection problems, the planner uses the last saved snapshot, and if there isn't one, it uses calm air to ensure repeatability of runs. Terrain information comes from the first available elevation source, queried in the following order: local GeoTIFF, OpenTopography Copernicus 30 m, Open-Meteo elevation, and finally flat terrain; all answers get cached to disk. OpenStreetMap building, exclusion zones, water bodies, and land cover information is fetched through the multi-endpoint Overpass client, which retries in case of connection problems. This information is cached on disk as GeoPackage files [3], and an index is built using the Shapely STRtree in a local metric coordinate system.

3.2.2 Lattice generation and node selection

Drones move through the airspace irrespective of the underlying road network; thus, the airspace is represented as a regular lattice structure. The area of interest is bounded by an $n \times n$ lattice in a local equirectangular projection system, whereby $n = \text{round}(box/spacing)$. A 6 km bounding box with 200 m spacing gives 900 candidate points. A candidate point becomes a node only if it meets a selection

rule based on four variables: DEM ground elevation, the tallest building in the cell, the legal ceiling, and restricted airspace. For node k with ground elevation z_k and tallest building b_k in its cell,

$$\mathrm{AGL}_k = \max\big(H_{\mathrm{cruise}},\, b_k + C\big), \qquad \mathrm{alt}_k = z_k + \mathrm{AGL}_k \tag{3.2}$$

with cruise height $H_{\mathrm{cruise}} = 60\,\mathrm{m}$ and clearance $C = 15\,\mathrm{m}$. A building's height is taken from its `height` tag; if there is none, from `building:levels` times 3 m; otherwise, from the default value for the building type. A node with $\mathrm{AGL}_k > 120\,\mathrm{m}$ is blocked, because it cannot be flown legally under the Drone Rules, 2021 [13]. A node inside a restricted polygon (an aerodrome, military land, a prison, a protected area or a government site) is removed. Each of the remaining nodes keeps its position within each coordinate system, its ground elevation, its building height, its cell's density and average height, its height above the ground, and its height above mean sea level.

3.2.3 Edge construction and pruning

Each surviving node is linked to its eight neighbours in both directions, so the graph is directed from the start. An edge is deleted if it intersects with the prohibited polygon, or if it fails the optional edge filtering hook (the simulation module performs its roof and terrain clearance test here). Then only the largest strongly connected component is kept; otherwise, the depot could become isolated on a subgraph.

Every residual edge retains its horizontal haversine distance d_h, its forward bearing θ, its slope $g = \Delta\mathrm{alt}/d_h$, and its 3D distance $d = \sqrt{d_h^2 + \Delta\mathrm{alt}^2}$.

3.2.4 Static risk terms

None of the static metrics depends on the weather, so they are computed only once per build. The edge is traversed at 25 m intervals along the line connecting terminal altitudes, and the metrics calculated include the flying altitude, the altitude of the ground surface, the highest roof below the edge, and the highest roof within 30 m of the edge. All eleven components of the risk, both static and depending on the weather, are listed in Table 3.1 together with their weight in the score

$$\rho_{\text{edge}} = \text{clip}\Big(\sum_k w_k \rho_k,\ 0,\ 1\Big), \qquad \rho(e;\omega) = \rho_{\text{edge}} \cdot \frac{d_h}{s} \tag{3.3}$$

where s is the lattice spacing. Multiplying by d_h/s turns the per-edge score into a cumulative exposure. The risk of a route therefore grows with the distance flown through risky air.

TABLE 3.1: Risk components (every $\rho_k \in [0,1]$) and their weights

Component	**Model**	w_k
Obstacle	Seeded obstacle discs, $0.9(1-r/R)$ inside a disc (synthetic graphs only)	1.00
Building	Mean over samples of [roof within 30 m] $\cdot\,\big(1-\text{clip}(m/60)\big)$, m = margin above that roof	0.60
Terrain	$\max\big(1-\text{clip}(m_{\min}/15),\ \text{clip}(\lvert g\rvert/0.30)\big)$: lowest clearance margin or sudden altitude change	0.30
Wind	$\text{clip}(CW/40)$, crosswind against a 40 km/h saturation speed	0.30
Gust	$\text{clip}\big(0.5\max(0, gust - v)/40\big)$	0.30
Weather	$0.5\,\text{clip}(rain/7.5) + 0.5\,\text{clip}(1 - vis/10\,\text{km})$, uniform over the graph	0.20

Component	**Model**	w_k
Population	Mean land-use class score along the edge (our values, ranked in the spirit of SORA [10]): assembly 1.0, commercial 0.7, major road 0.6, residential 0.5, industrial 0.3	0.50
No-fly buffer	$\max_k \exp(-\delta_k/150\,\mathrm{m})$ to the nearest restricted zone, zero beyond 600 m	0.80
Water	Fraction of samples over water (risk of losing the payload)	0.30
Comm/GPS	$\mathrm{clip}(\mathit{density}/0.30)\cdot\mathrm{clip}(\bar{h}/30\,\mathrm{m})$, urban canyon effect	0.20
Turbulence	$\mathrm{clip}(\mathit{density}/0.30)\cdot\mathrm{clip}(v/40)$	0.20

The weights are design values that put restricted zones, buildings and crowds ahead of the weather terms. They have not been calibrated against flight data, and calibrating them is part of the evaluation work that remains in Phase I.

3.2.5 Wind-aware energy model

Take an edge on bearing θ_{edge} and a wind of speed v coming from θ_{from} (meteorological convention). Its headwind and crosswind components are

$$HW = v\cos(\theta_{\text{edge}} - \theta_{\text{from}}), \qquad CW = v\,|\sin(\theta_{\text{edge}} - \theta_{\text{from}})| \tag{3.4}$$

Positive HW means headwind. We write the edge's energy proxy as

$$\varepsilon = d\left(1 + |g| + \beta_h \frac{\max(0, HW)}{V_c} + \beta_c \frac{CW}{V_c}\right) \tag{3.5}$$

with cruise airspeed $V_c = 36\,\mathrm{km/h}$, $\beta_h = 0.40$ and $\beta_c = 0.15$. Tailwind is not credited intentionally to make sure that $\varepsilon \geq d$ for all edges, a requirement for the Tier 1 heuristic. For wind speed, 80 m prediction is used wherever it is available,

as 80 m is near the 60–120 m cruise band. Otherwise, the 10 m value is increased to the cruising altitude using the power law

$$v_h = v_{10}\left(\frac{h}{10}\right)^{\alpha}, \qquad \alpha = 0.25 \tag{3.6}$$

from Peterson and Hennessey [17], and the gust factor measured at 10 m is then applied to this flight-height wind.

Equation (3.4) makes the edge costs directional. In wind, the transitions $u \to v$ and $v \to u$ have different costs, and hence even a symmetric lattice needs a directed graph.

3.2.6 Weather-dependent risk and the go/no-go gate

Only four of the eleven risk elements are sensitive to weather: cross-wind, gust excess, turbulence, and uniform rain/visibility element. Cross-wind is determined by edge heading, resulting in corridor-specific risk differences, and rotating the wind direction without changing its strength can move the low-risk zone of the front on the map. The function `apply_weather(G, ω)` re-calculates the costs of edges based on their static data, meaning that it is unnecessary to re-construct the spatial layers for the new snapshot.

This is followed by the weather gate. A mission is no-go when the WMO index shows a thunderstorm (95–99), precipitation intensity reaches 7.5 mm/h, visibility is less than 1 km, and/or gusts are more than 45 km/h, which is the provisional limit for an aircraft of the class F450. The reasons go onto the graph, and while any of them is present Tier 2 will not pick a route.

Algorithm 1 Lattice airspace graph construction and costing (Module 1)

Require: Region, box size, spacing s; DEM; buildings $\mathcal{B}$; restricted zones $\mathcal{Z}$; weather ω; weights w_k; optional `edge_filter`

Ensure: Directed graph $G = (V, E)$ with costs $\mathbf{c}(e) = [d, \varepsilon, \rho]$ and no-go reasons $\mathcal{R}$

$n \leftarrow \mathrm{round}(box/s)$; place an $n \times n$ lattice in a local equirectangular frame
for each lattice point k **do** ▷ node selection
  $z_k \leftarrow$ DEM elevation; $b_k \leftarrow$ tallest building in the cell of k (from $\mathcal{B}$)
  $\mathrm{AGL}_k \leftarrow \max(H_{\text{cruise}}, b_k + C)$; $\mathrm{alt}_k \leftarrow z_k + \mathrm{AGL}_k$
  if $\mathrm{AGL}_k \le 120\,\text{m}$ **and** k lies outside every polygon in $\mathcal{Z}$ **then** add k to V
  end if
end for
for each $u \in V$ and each of its eight lattice neighbours $v \in V$ **do** ▷ edge construction
  if segment uv crosses a polygon in $\mathcal{Z}$ **or** `edge_filter`(u, v) rejects it **then continue**
  add the directed edges (u, v) and (v, u) to E
end for
keep only the largest strongly connected component of G
for each edge $e = (u, v) \in E$ **do** ▷ static features and static risk
  $d_h \leftarrow$ haversine length; $\theta \leftarrow$ forward azimuth; $g \leftarrow \Delta\mathrm{alt}/d_h$; $d \leftarrow \sqrt{d_h^2 + \Delta\mathrm{alt}^2}$
  sample e every 25 m; read flight altitude, ground height, roof below and roof within 30 m
  store ρ_k for building, terrain, population, no-fly buffer, water and comm/GPS
end for
$\mathcal{R} \leftarrow$ reasons among: WMO code 95–99, rain ≥ 7.5 mm/h, visibility < 1 km, gust > 45 km/h
$v \leftarrow 80$ m wind forecast, else $v_{10}(h/10)^{\alpha}$ ▷ this part re-runs per ω
for each edge $e \in E$ **do**
  $HW \leftarrow v\cos(\theta - \theta_{\text{from}})$; $CW \leftarrow v\,|\sin(\theta - \theta_{\text{from}})|$
  $\varepsilon \leftarrow d\left(1 + |g| + \beta_h \max(0, HW)/V_c + \beta_c CW/V_c\right)$ ▷ tailwind earns no credit
  compute ρ_k for wind, gust, weather and turbulence
  $\rho_{\text{edge}} \leftarrow \mathrm{clip}\left(\sum_k w_k \rho_k, 0, 1\right)$; $\rho \leftarrow \rho_{\text{edge}}\, d_h/s$; $\mathbf{c}(e) \leftarrow [d, \varepsilon, \rho]$
end for
store $\mathcal{R}$ on G; **return** G

3.3 TIER 1 PARETO FRONT ENGINE (MODULE 2)

The Module 2 calculates the set of all non-dominated paths from a specific source to target. Using the costed graph from Module 1, the two end nodes and the expansion threshold, this module outputs the Pareto set $\mathcal{P}$ in the form of vectors with associated node paths. It also gives some diagnosis data such as if the expansion threshold stopped the procedure, how many labels were expanded and created, and the total computation time.

3.3.1 Labels and dominance

Our algorithm uses NAMOA* [11]. The partial path is represented by the label $(node, \mathbf{g}, parent)$ in one flat table, while the full paths are reassembled after-the-fact using parent references. To avoid duplicating the path at each push operation—a task which becomes too expensive once we have thousands of labels—in our table we keep only the reference. A newly created label $\mathbf{g}'$ at the vertex v is thrown out if there is already a label at v that weakly dominates $\mathbf{g}'$, namely is less or equal to $\mathbf{g}'$ for each objective with a certain tolerance. Identical vectors are considered dominated, so no duplicate reaches the front. The labels that are dominated by the new arrival are labeled as dead and skipped on extraction. There is no need for the visited set since any loop will be dominated by an earlier label at the same node because all edges have positive lengths.

3.3.2 Admissible vector heuristic and pruning

The heuristic function for vertex v can be expressed as

$$\mathbf{h}(v) = [h,\, h,\, 0] \tag{3.7}$$

where h refers to the great-circle distance between vertex v and the target vertex. The heuristic is admissible with respect to all the three criteria. For distance, all edges fulfill the condition $d \geq d_h$, and the total horizontal distance traveled to reach the target vertex is always equal to or greater than h. For energy, Equation (3.5) gives $\varepsilon \geq d$. On risk, $\rho \geq 0$. A label will be pruned when the current front weakly dominates $\mathbf{g} + \mathbf{h}$. This check occurs at both generation and extraction of the label, because the front might improve in between. Why do we sort the open list by $\|\mathbf{g} + \mathbf{h}\|$? Efficient paths finish early, and each efficient path improves the bound on pruning.

3.3.3 Truncation and front thinning

A ceiling on expansion avoids the problem of an unrestricted search. This solution is labeled as being truncated if at least one active label has the ability to improve the front. An optional ε-dominance phase can cap very large fronts at N paths while keeping the three single-objective extreme points. We leave it off by default, so the benchmarks use the exact front.

Algorithm 2 shows how the entire process is carried out, where $\preceq$ represents weak dominance.

Algorithm 2 NAMOA* with a label table (Tier 1)

Require: Graph $G = (V, E)$, source s, target t, costs $\mathbf{c}(u, v) = [d, \varepsilon, \rho]$, cap M

Ensure: Pareto front $\mathcal{P}$, flag *truncated*

$L \leftarrow [(s, [0,0,0], -1)]$; $at[s] \leftarrow \{0\}$; $F \leftarrow \emptyset$; $Open \leftarrow \{(\|\mathbf{h}(s)\|, 0)\}$
while $Open \neq \emptyset$ **do**
  if $expanded \geq M$ **then**
    $truncated \leftarrow \exists$ live $\ell \in Open$ with $F \not\preceq \mathbf{g}_\ell + \mathbf{h}(u_\ell)$; **break**
  end if
  $\ell = (u, \mathbf{g}) \leftarrow$ POPMIN($Open$)
  if ℓ is dead **or** $F \preceq \mathbf{g} + \mathbf{h}(u)$ **then continue**
  $expanded \leftarrow expanded + 1$
  for each successor v of u **do**
    $\mathbf{g}' \leftarrow \mathbf{g} + \mathbf{c}(u, v)$
    if $\exists o \in at[v] : \mathbf{g}_o \preceq \mathbf{g}'$ **then continue** ▷ node dominance
    if $F \preceq \mathbf{g}' + \mathbf{h}(v)$ **then continue** ▷ prune on $g + h$
    mark dead every $o \in at[v]$ with $\mathbf{g}' \preceq \mathbf{g}_o$
    append label $(v, \mathbf{g}', \ell)$ to L and to $at[v]$
    if $v = t$ **then**
      $F \leftarrow \{f \in F : \mathbf{g}' \not\preceq f\} \cup \{\mathbf{g}'\}$
    else
      push $(\|\mathbf{g}' + \mathbf{h}(v)\|, \text{label})$ onto $Open$
    end if
  end for
end while
rebuild each route in F by following parent ids; **return** $\mathcal{P}$, *truncated*

3.3.4 Why an exact method

However, on a large graph, population-based approaches tend to be more efficient. Still, we decided to keep using the exact search algorithm. The graph and weather conditions always provide the same result. Interior routes, which mixed payloads need, are the ones a stochastic archive tends to lose (Section 4.4.1). And Tier 1 runs once per region and weather window.

3.3.5 Pareto front store and refresh

The front is reused across missions, so the system needs a rule for when reuse is safe. The front store saves one JSON record per (region, source, target) with the front, a SHA-256 hash of the static graph (never the weather-dependent costs), the weather snapshot it was built for, a validity limit three hours after creation, the truncation flag and the search diagnostics.

For each request, the store lists why the cache entry cannot be reused, such as no record present, a forced cache refresh, new static graph, different identity, expiration, or weather drift. Drift means the wind speed has moved by more than 10 km/h, the direction has turned by more than 45° while the wind is above 10 km/h, the gust has moved by more than 15 km/h, or the go/no-go state has flipped. With no reason, Tier 2 runs against the stored front and no search happens. With a reason, only `apply_weather` and Tier 1 run again, and the spatial layers stay cached.

3.4 BASELINE PLANNERS (MODULE 3)

Module 3 holds the planners that Tier 1 and Tier 2 are compared against. They read the same costed graph, so any difference in the results comes from the search method and not from the data.

Three single-objective A* searches minimise distance, energy and risk separately; the distance search uses the haversine heuristic and the other two a zero heuristic. Each returns one extreme point of the front. A weighted-sum A* scalarises the cost vector with a preset's Tier 2 weights, $c_w(e) = \sum_j w_j c_j(e)/s_j$, where s_j is the spread of objective j across the three extreme routes. This is the classic approach our problem statement criticises, and it can reach only supported points on the convex hull of the front. The population baseline adapts MOPSO [5] to graph routes: a particle is a valid path, the swarm is seeded with the extreme routes and biased random walks, and in each iteration a particle follows its personal best or an archive leader for part of the way and is repaired to the target with a weighted shortest suffix. The archive uses the same dominance helper as Tier 1. Report-scale settings are 100 particles, 200 iterations, an archive of 100 and 30 seeded repetitions.

3.5 TIER 2 PAYLOAD METADATA INTERPRETER (MODULE 4)

Module 4 turns a description of the payload into a route. Its inputs are the stored front, the weather go/no-go reasons from the graph and the four tags $(F, W, U, P_{\text{lvl}})$.

Its output is a selection record: the tags, the derived weights, the selected route's index, cost vector and waypoints, the normalised scores, its rank on each objective, whether it is an extreme or non-extreme point, the latency, the number of battery-feasible routes and, when no route can be flown, a HOLD advisory. The record is written to JSON and CSV. A tag outside its integer range raises an error before any weight is derived, so a typing mistake cannot quietly change the route.

3.5.1 Semantic mapping

The bridge between the tiers maps each payload tag to exactly one Tier 1 objective, as Table 3.2 shows.

TABLE 3.2: Semantic mapping from payload tags to Tier 1 objectives

Payload tag	**Range**	**Tier 1 objective**	**Reasoning**
Fragility class F	1–5	w_{risk} (flight risk)	Fragile items should avoid turbulent, exposed and crowded corridors
Payload weight class W	1–3	w_{energy} (energy)	A heavier payload raises hover and lift power, so energy matters more
Delivery urgency U	1–3	w_{dist} (distance)	Time-critical deliveries need the shortest path
Priority level P_{lvl}	1–5	Amplifier	Strengthens the dominant weight for high-priority missions

3.5.2 Weight derivation

The three weights are determined from the tags without expert intervention:

$$r_{\text{raw}} = 2F, \qquad d_{\text{raw}} = 3U, \qquad e_{\text{raw}} = 2W, \qquad S = r_{\text{raw}} + d_{\text{raw}} + e_{\text{raw}} \tag{3.8}$$

$$w_{\text{risk}} = \frac{r_{\text{raw}}}{S}, \qquad w_{\text{dist}} = \frac{d_{\text{raw}}}{S}, \qquad w_{\text{energy}} = \frac{e_{\text{raw}}}{S} \tag{3.9}$$

The urgency weight gets a multiplier of 3, whereas the remaining two weights get a multiplier of 2 since urgency is considered the most delicate among the three parameters in medical delivery. Energy is associated with the payload weight because the bigger the load, the greater the power needed for hovering and ascending; whereas, no such direct association exists between weight and risk/distance.

All three raw scores play a role in every weight score. An increase in F increases w_{risk} and simultaneously decreases the other two weights, meaning that a change in just one tag affects all the weight scores. This is the reason the formula is not a lookup table. There is no fixed list of payload types, and any of the $5 \times 3 \times 3 \times 5 = 225$ tag combinations gives a valid, different weighting.

For high-priority missions ($P_{\text{lvl}} \geq 4$), the largest weight is amplified:

$$w_{\max} \leftarrow w_{\max} + 0.1\, w_{\max}, \qquad w_j \leftarrow \frac{w_j}{\sum_k w_k} \tag{3.10}$$

A flat bonus added to every score was rejected, because it never changes which route scores highest.

3.5.3 Utility scoring and selection

For each Pareto point p_i with objective values $(d_i, \varepsilon_i, \rho_i)$, the values are min-max normalised over the whole front,

$$\tilde{d}_i = \frac{d_i - d_{\min}}{d_{\max} - d_{\min}}, \quad \tilde{\varepsilon}_i = \frac{\varepsilon_i - \varepsilon_{\min}}{\varepsilon_{\max} - \varepsilon_{\min}}, \quad \tilde{\rho}_i = \frac{\rho_i - \rho_{\min}}{\rho_{\max} - \rho_{\min}} \tag{3.11}$$

so that 0 is the best value on the front and 1 the worst. A degenerate objective, where all routes are equal, normalises to zero. The utility score is

$$\text{score}(p_i) = w_{\text{dist}}(1-\tilde{d}_i) + w_{\text{energy}}(1-\tilde{\varepsilon}_i) + w_{\text{risk}}(1-\tilde{\rho}_i) \tag{3.12}$$

and the mission route is $p^* = \arg\max_{p_i \in \mathcal{P}} \text{score}(p_i)$. Ties go to the lower index, so the result is deterministic.

The argmax runs over the full front, so Tier 2 can land anywhere on it, including between the extremes. For a payload with mixed needs the best-scoring route is often an interior one that no single-objective search would return.

3.5.4 Reference presets

Table 3.3 gives the weights the formula produces for four reference presets. Only emergency medicine has $P_{\text{lvl}} \geq 4$, so it is the only preset whose dominant weight is amplified.

TABLE 3.3: Reference payload presets and their derived weights

Preset	F	W	U	P_{lvl}	w_{risk}	w_{dist}	w_{energy}
Fragile medical sample	5	1	1	3	0.667	0.200	0.133
Emergency medicine	1	1	3	5	0.144	0.712	0.144
Disaster relief	1	3	1	2	0.182	0.273	0.545
Mixed cargo	3	2	2	3	0.375	0.375	0.250

As a worked example, take emergency medicine. The raw scores are $r_{\text{raw}} = 2$, $d_{\text{raw}} = 9$ and $e_{\text{raw}} = 2$, which sum to 13, so the weights before amplification are 0.154, 0.692 and 0.154. The priority is 5, so the distance weight grows by a tenth to 0.762. After renormalising by the new total of 1.069, the weights are $w_{\text{risk}} = 0.144$, $w_{\text{dist}} = 0.712$ and $w_{\text{energy}} = 0.144$, which are the values in the table.

3.5.5 Battery feasibility

The payload weight class also decides whether a route can be flown at all. Each class maps to a nominal payload mass m_W (0.25, 0.5 and 1.0 kg for $W = 1,2,3$), and a route's energy proxy is converted to watt-hours with

$$E_{\mathrm{Wh}} = k_{\mathrm{cal}}\,\varepsilon \left(\frac{m_{\mathrm{uav}} + m_W}{m_{\mathrm{uav}}} \right)^{1.5} \tag{3.13}$$

where the exponent shows the relationship between rotor power and weight. With the current values ($k_{\mathrm{cal}} = 6$ Wh per unit of energy proxy, $m_{\mathrm{uav}} = 1.5$ kg, a 74 Wh battery and a 20% reserve), a route is feasible when it needs at most 59.2 Wh. Routes failing to meet this condition are then removed before taking the argmax. The constants are noted as placeholders in the code and their actual values, calculated through simulation, will replace them.

3.5.6 Weather hold and decision record

When the weather gate gives any reason, or the battery rules out every path, Tier 2 returns a HOLD advice with the reason instead of a path. In all other situations, the feasible path with the best score is provided.

Algorithm 3 gives the whole process. No search operations are performed at this point. It needs only one scan through the saved front and takes just a few milliseconds.

Algorithm 3 Payload-aware route selection on the stored front (Tier 2)

Require: Front $\mathcal{P} = \{(\mathbf{c}_i, \pi_i)\}_{i=1}^{K}$, $\mathbf{c}_i = [d_i, \varepsilon_i, \rho_i]$; no-go reasons $\mathcal{R}$; tags $(F, W, U, P_{\text{lvl}})$; m_W, m_{uav}, k_{cal}, E_{bat}, reserve η

Ensure: Mission route π^* and its selection record, or a HOLD advisory

if $F, P_{\text{lvl}} \notin \{1,\dots,5\}$ **or** $W, U \notin \{1,2,3\}$ **then raise** an error ▷ before any weight is derived

$r_{\text{raw}} \leftarrow 2F$; $d_{\text{raw}} \leftarrow 3U$; $e_{\text{raw}} \leftarrow 2W$; $S \leftarrow r_{\text{raw}} + d_{\text{raw}} + e_{\text{raw}}$

$w_{\text{risk}} \leftarrow r_{\text{raw}}/S$; $w_{\text{dist}} \leftarrow d_{\text{raw}}/S$; $w_{\text{energy}} \leftarrow e_{\text{raw}}/S$

if $P_{\text{lvl}} \geq 4$ **then** ▷ priority amplifier

  $w_{\max} \leftarrow 1.1\, w_{\max}$ for the largest of the three weights

  divide every weight by $w_{\text{risk}} + w_{\text{dist}} + w_{\text{energy}}$

end if

for each objective $j \in \{d, \varepsilon, \rho\}$ **do** ▷ front-wide ranges

  $lo_j \leftarrow \min_i c_{i,j}$; $hi_j \leftarrow \max_i c_{i,j}$

end for

$E_{\text{use}} \leftarrow E_{\text{bat}}(1-\eta)$; $\mathcal{F} \leftarrow \emptyset$

for $i = 1$ **to** K **do**

  for each objective j **do** ▷ min–max normalisation

    $\tilde{c}_{i,j} \leftarrow 0$ if $hi_j = lo_j$, else $(c_{i,j} - lo_j)/(hi_j - lo_j)$

  end for

  $s_i \leftarrow w_{\text{dist}}(1 - \tilde{d}_i) + w_{\text{energy}}(1 - \tilde{\varepsilon}_i) + w_{\text{risk}}(1 - \tilde{\rho}_i)$

  $E_{\text{Wh},i} \leftarrow k_{\text{cal}}\, \varepsilon_i \left((m_{\text{uav}} + m_W)/m_{\text{uav}}\right)^{1.5}$ ▷ battery proxy

  if $E_{\text{Wh},i} \leq E_{\text{use}}$ **then** $\mathcal{F} \leftarrow \mathcal{F} \cup \{i\}$

  end if

end for

if $\mathcal{R} \neq \emptyset$ **then**

  return HOLD, with the weather reasons in $\mathcal{R}$

else if $\mathcal{F} = \emptyset$ **then**

  return HOLD, no route fits E_{use} at payload class W

end if

$i^* \leftarrow \arg\max_{i \in \mathcal{F}} s_i$, ties to the lower index ▷ full front, not only extremes

$rank_j \leftarrow$ position of $c_{i^*,j}$ among the sorted values of objective j

type $\leftarrow$ *extreme* if i^* is best on any objective, else *non-extreme*

wp $\leftarrow$ (latitude, longitude, altitude) of every node on π_{i^*}

write tags, weights, i^*, $\mathbf{c}_{i^*}$, *wp*, $\mathbf{s}$, *rank*, *type*, $|\mathcal{F}|$ and latency to JSON and CSV

return π_{i^*} and the selection record

3.6 EVALUATION AND BENCHMARKING (MODULE 5)

Module 5 considers two factors: (1) the quality of the front, and (2) the way in which Tier 2 makes its choices. Every measure is computed per scenario and written to CSV and JSON, so passes can be compared.

3.6.1 Front quality measures

There is no known true front for our graphs, so the reference front $\mathcal{R}$ of a scenario is the non-dominated union of every route found by every method and repetition, and all measures are computed in the space normalised by its ideal and nadir points. The hypervolume (HV) [22] is the volume dominated by a front and bounded by a reference point, here the normalised nadir padded by 10%, $[1.1, 1.1, 1.1]$. It is computed exactly in three dimensions by slicing along the first objective. Larger is better. The inverted generational distance (IGD) measures how well a front A covers the reference front:

$$\mathrm{IGD}(A, \mathcal{R}) = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} \min_{a \in A} \|r - a\|_2 \tag{3.14}$$

Smaller is better, and zero means that A contains every reference point. The spread Δ measures how evenly the points are distributed and how far they reach toward the extremes of $\mathcal{R}$; smaller is better. Runtime and front size are also recorded.

3.6.2 Tier 2 evidence

We do not report a Tier 2 “accuracy”, as no ground-truth label says which route is correct for a payload. For each preset the module records the selected route’s rank on each objective, whether it is an extreme or non-extreme point, the latency, and a rank-based payload-alignment score

$$A = \sum_{k \in \{d,\varepsilon,\rho\}} w_k \left(1 - \frac{\mathrm{rank}_k - 1}{K - 1}\right) \tag{3.15}$$

where K is the front size. Because A uses ranks, it does not depend on the min-max utility that Tier 2 itself maximises. The same score is computed for four competing selection rules on the same front: the shortest route, the knee point, the largest NSGA-II crowding distance [6, 18], and a fixed equal-weight utility in the style of [4].

3.6.3 Tag sweep and monotonicity

The sweep performs Tier 2 for all 225 tag combinations, counting the different routes chosen and the non-extreme choices, and testing monotonicity. With all other tags constant, increasing F must not lead to a riskier route, increasing U to a longer route, or increasing W to more energy.

3.6.4 The evaluation cycle

Benchmarking is performed as a loop operation. Scenarios are constant; each algorithm is tested; fronts are assessed, and Tier 2 solutions are studied; routes are

flown through simulation; problems are evaluated; the model is updated, and the loop starts anew. One iteration covers four graphs with nine wind conditions, with MOPSO at 100 particles, 200 iterations and 30 runs, about 21.6 million particle iterations.

3.7 MISSION EXPORT AND DASHBOARD (MODULE 6)

3.7.1 Mission file generation

Module 6 stores the selected path as QGC WPL 110 and QGroundControl `.plan` files. It merges adjacent waypoints having same bearing and slope values. The waypoints of the path are specified in frame 3 (`GLOBAL_RELATIVE_ALT`), relative to home. Therefore, the altitude relative is calculated by the exporter as

$$alt_{\text{rel}} = alt_{\text{AMSL}} - H_{\text{home}} \tag{3.16}$$

where H_{home} is the ground altitude below the first waypoint. The altitude above the ground is never written into frame 3 because the ground level varies.

3.7.2 Operator dashboard

A small Tkinter dashboard lets an operator load a stored graph and front, type the four tags, see the route on a map and export the mission. Two defects are open: it

does not yet pass the weather reasons to Tier 2, and it can load a graph and a front from different runs.

3.8 SIMULATION AND SITL VALIDATION (MODULE 7)

A route that scores well on the graph may still be hard to fly, so Module 7 checks the flight itself. It flies the Tier 2 route through a 3-D model of the delivery area in Gazebo Harmonic [16], with ArduCopter running in software-in-the-loop [1], and compares the flown track with the plan. Communication with the autopilot uses MAVLink [12]. Figure 3.3 shows the pipeline.

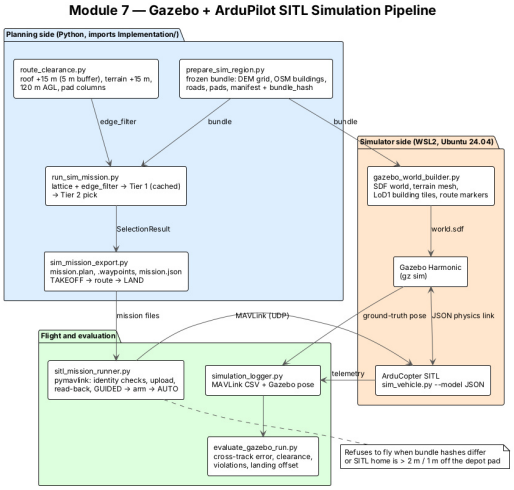


FIGURE 3.3: Gazebo and ArduPilot SITL simulation pipeline

3.8.1 Bundle, clearance and frame contract

A region is first frozen into a bundle: a terrain grid, OSM buildings with heights, roads, the depot and destination pads, and a manifest. Its hash is written into

the graph, the front, the mission file and the world, and the runner refuses to fly if they do not match. Before NAMOA* runs, a clearance checker passed in as `edge_filter` removes every edge that passes less than 15 m above a buffered roof or the terrain, or goes above 120 m above ground. The Gazebo world's origin is the depot pad, where SITL also places home, so altitudes agree everywhere.

3.8.2 Mission sequence

Figure 3.4 follows one mission from tags to flown route. Tier 1 runs only when the stored front is stale or missing. Tier 2 then returns a HOLD advisory or a selection exported as a mission file. The runner uploads and reads back the mission, arms, switches to AUTO and logs telemetry until landing. Afterward, the evaluator assesses cross-track error, minimum clearance, rules infringements, and landing offset.

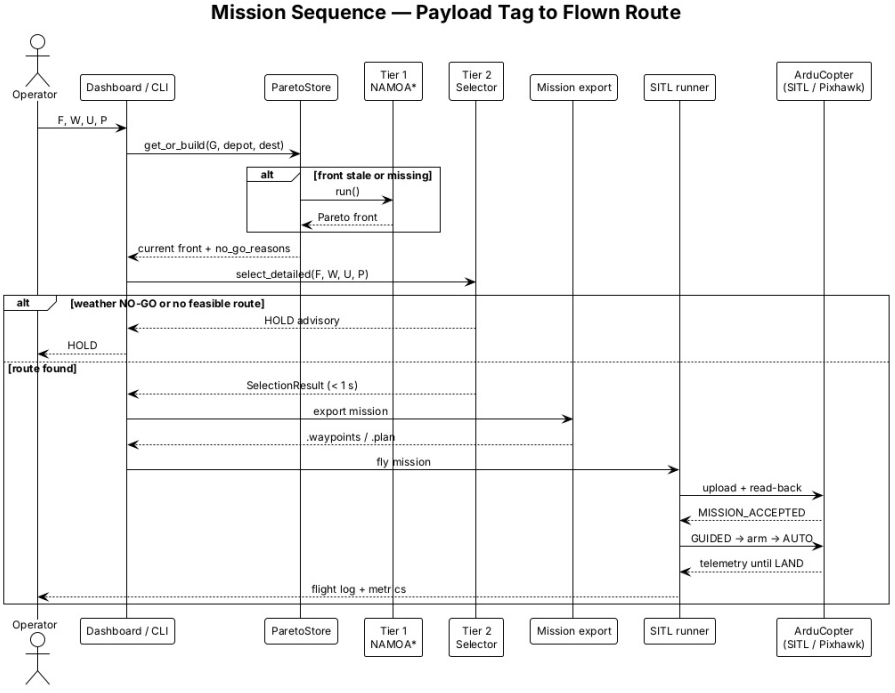


FIGURE 3.4: Mission sequence from payload tag entry to the flown route

CHAPTER 4

EXPERIMENTAL RESULTS AND PERFORMANCE ANALYSIS

This chapter contains the information that forms the basis of the system, both the hardware and software, the experiments carried out thus far, and the conclusions made from them. The quantitative results come from four different places: the first run of the evaluation loop (refer Section 3.6), wind experiment, the ablation runner, and the very first SITL flights. They are preliminary in a specific sense: MOPSO was run at reduced settings, the risk weights and battery constants are not yet calibrated, and the synthetic density labels have a known defect. Each caveat is stated next to the result it affects.

4.1 DATASET DESCRIPTION

The datasets of the works we reviewed are custom or not released, so, as is common in this field, we built our own environments. There are two kinds: synthetic airspace graphs, generated from fixed seeds, for controlled benchmarks; and a real-region lattice built from public geographic and weather data. Table 4.1 lists every graph used in this chapter.

4.1.1 Synthetic airspace graphs

The synthetic graphs come from the seeded builder of Section 3.2. Each is a jittered grid of about 1.3 km spacing with 8-neighbour directed edges, seeded Gaussian

TABLE 4.1: Graphs and scenarios used in the experiments

Scenario	Size	Description
S1 small sparse	50 nodes	Synthetic grid, seed 101, 5% obstacle coverage label, weather seeded
S2 medium	196–200 nodes	Synthetic grid, seed 202, 20% label; weather seeded (benchmark) or set explicitly (wind experiment)
S3 large dense	500 nodes	Synthetic grid, seed 303, 40% label, weather seeded
Real-region lattice	893 nodes, 6,752 edges	6 km box, 200 m spacing, offline fixture layers (ablation)
Small lattice	2 km box, 200 m spacing	Lattice used for the route figure in Section 4.4.5
Flat lattice	225 nodes, 1,624 edges	3 km box, 200 m spacing, no layers (directional cost check)
Simulation bundle	1.5 km box, 100 m lattice	Frozen terrain, buildings, roads and pads for Gazebo and SITL

hills for terrain, obstacle discs, and seeded wind between 12 and 28 km/h with a gust factor of 1.4. The three scales (50, 200 and 500 nodes) were chosen to cover a sanity check, a mid-sized core case and a stress case. Every draw comes from the seed, so the graphs are reproducible.

There is one known defect in this data. The density obstacle labels (5%, 20% and 40%) do not affect the graph builder at all; rather, the benchmark runner rescales the end risk value. As a result, the scenarios vary in size and seeds and not in obstacle density regardless of the labels. This is corrected in the next pass, and all benchmark values are recalculated.

4.1.2 Real-region lattice

Four public data sources feed the real-region lattice (Table 4.2). Nothing in the method ties it to one place; our example is a coastal urban neighbourhood. The default 6 km box at 200 m spacing has 900 candidate nodes, of which 893 remain

after node selection and connectivity pruning in the fixture run, linked by 6,752 directed edges.

TABLE 4.2: Environment data sources for the real-region lattice

Layer	**Source**	**Used for**
Weather	Open-Meteo forecast [15]	Wind speed and direction at 10 m and 80 m, gust, rain, visibility, WMO code
Terrain	Local GeoTIFF, OpenTopography Copernicus 30 m, Open-Meteo elevation (Copernicus 90 m), flat fallback	Ground elevation of nodes and edge samples
Buildings	OpenStreetMap via Overpass	Footprints and heights; node altitude, building risk, density, turbulence, urban canyon
Restricted zones	OpenStreetMap via Overpass	Aerodromes, military land, prisons, protected areas, government sites; node and edge removal, soft buffer
Water and land use	OpenStreetMap via Overpass, coastline	Water fraction, population exposure classes, major roads

Public Overpass server sometimes resets or times out. For the offline and replicable experiments, the project offers fixture layers that represent manually created test data mimicking the real region. The ablation from Section 4.4.4 was done using such fixtures showing how each layer affects the planner rather than providing metrics available in reality.

4.1.3 Payload tag space and simulation bundle

We evaluate Tier 2 on all 225 tag combinations. Four reference presets (Table 3.3) stand in for deliveries led by risk, by distance and by energy, and one balanced delivery. For simulation we freeze a 1.5 km region into a bundle, and its hash binds the planner graph, stored front, mission file and Gazebo world to each other.

4.2 ECOSYSTEM

4.2.1 Hardware configuration

All planner experiments in this chapter ran on a laptop-class CPU without a GPU. The planner does not need one: Tier 1 is a graph search that runs on a single core, and Tier 2 is a closed-form computation. The simulation runs under WSL2 (Ubuntu 24.04) on the same class of machine.

Phase II moves the system onto standard, commercially available hardware: an F450 quadcopter frame with a Pixhawk flight controller running ArduCopter, a Raspberry Pi 5 companion computer that hosts Tier 2 on board, a servo payload release and a MAVLink telemetry link to the ground station.

4.2.2 Software environment

All code is written in Python 3. NetworkX [8] holds the directed airspace graph and runs the A* baselines; NumPy handles dominance checks and metrics; Shapely 2 STRtree indexes hold the spatial layers; and GeoPandas, pyogrio and OSMnx [3] parse and cache the OpenStreetMap layers. Figures use Matplotlib, the dashboard Tkinter, and the offline test suites unittest. Simulation uses Gazebo Harmonic [16], ArduPilot SITL [1], pymavlink over MAVLink [12] and QGroundControl.

4.3 EXPERIMENTS CONDUCTED

Table 4.3 summarises the experiments. Each is described below, and Section 4.4 analyses the results.

TABLE 4.3: Experiments carried out in Phase I so far

No.	Experiment	Data	Question
E1	Front quality benchmark	S1, S2, S3	Does the exact Tier 1 front beat single-objective A* and MOPSO on HV, IGD and spread, and at what runtime cost?
E2	Wind direction study	S2 under calm air and 8 directions; 15° sweep	How much does wind direction alone change the front and the Tier 2 picks?
E3	Directional edge costs	Flat lattice, wind from the east	Do edge costs respond to heading as the model predicts?
E4	Risk-layer ablation	Real-region lattice, fixture layers	Which layers drive risk, energy and front size?
E5	Tier 2 selection	S2, S3 and small lattice fronts	Do different payloads get different routes, including non-extreme ones, within 1 s?
E6	SITL flights	Fixture bundle, ArduCopter SITL	Is the mission chain from Tier 2 to the autopilot correct?
E7	Automated tests	Planner and simulation suites	Do the components behave as specified?

The settings were as follows. In E1, all methods used the same source and target per scenario, and HV, IGD and spread were computed against the union reference front. MOPSO ran at a reduced setting of 24 particles, 30 iterations and 2 repetitions per scenario to get a complete first pass quickly; the report-scale run (100, 200 and 30) is the next step. In E2, the S2 graph (196 nodes, 1,404 edges, seed 202) was costed under calm air and under a 25 km/h wind with 35 km/h gusts from each of eight compass directions, then in 15° steps, with the graph, layers, source and target unchanged. E3 used a flat 3 km lattice with no terrain, buildings or zones under a 25 km/h east wind. E4 rebuilt the real-region front

with all layers, without wind, without terrain and without buildings, under a 30 km/h wind from the south-west. E5 ran the four presets on the S2 front (16 routes), the S3 front (115 routes) and a 2 km lattice. E6 flew planner missions from a fixture bundle against ArduCopter SITL (4.8.0-dev, built-in quadcopter physics, five times real time), and a Tier 2 mission was also flown in the Gazebo city world. E7 covers about 220 planner tests and 45 simulation tests, all offline; Appendix B lists them. One of them compares the NAMOA* front with a brute-force front over all simple paths on a small graph, which is our strongest check that Tier 1 is exact.

4.4 PERFORMANCE ANALYSIS

4.4.1 Tier 1 front quality against the baselines

The first-pass benchmark is in Table 4.4.

NAMOA* achieves a perfect zero inverted generational distance (IGD) for all three cases, with all reference points found on the front of NAMOA*. This is the best that can be achieved by an exact algorithm and indicates clearly that no solution is lost through pruning rules. MOPSO recovers 12 of the 16 routes on S2 and 38 of the 115 on S3, and its HV is close to that of NAMOA*, with a gap of 0.0004 on S2 and 0.009 on S3.

The HV gap is small because MOPSO's archive still reaches the extremes of the front while missing most interior points, and interior points are the ones Tier 2 needs for mixed payloads. For this project the IGD and front-size columns say

TABLE 4.4: First-pass benchmark on the synthetic scenarios (HV higher is better; IGD and Δ lower are better)

Scenario	Method	Front	Time (s)	HV	IGD	Δ
S1 (50)	NAMOA* (Tier 1)	2	0.003	0.131	0.000	0.000
	A* distance	1	<0.001	0.121	0.866	–
	A* energy	1	<0.001	0.121	0.866	–
	A* risk	1	<0.001	0.011	0.866	–
	MOPSO	2	0.363	0.131	0.000	0.000
S2 (200)	NAMOA* (Tier 1)	16	0.162	1.2065	0.000	1.001
	A* distance	1	<0.001	0.107	1.021	–
	A* energy	1	<0.001	1.083	0.648	–
	A* risk	1	<0.001	0.011	0.887	–
	MOPSO	12	1.505	1.2061	0.017	1.027
S3 (500)	NAMOA* (Tier 1)	115	33.761	1.1235	0.000	0.666
	A* distance	1	0.001	0.062	0.861	–
	A* energy	1	0.002	0.674	0.442	–
	A* risk	1	0.002	0.013	1.116	–
	MOPSO	38	4.326	1.1146	0.043	0.814

more than HV. NAMOA* also spreads its points more evenly, with Δ of 0.666 against 0.814 on S3. On S1 both methods find the same two-route front, which tells us the smallest scenario is too easy to separate them.

The single-objective A* routes reach a high HV only when their one objective happens to dominate the volume, as A* energy does on S2. None of them can cover the front, and their IGD stays between 0.44 and 1.12. Figure 4.1 plots HV per method.

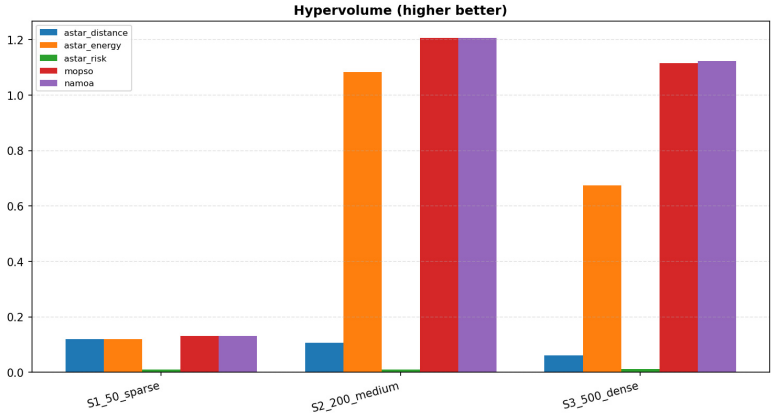


FIGURE 4.1: Hypervolume per method and scenario (first pass)

Exactness has a price. On S3, NAMOA* took 33.8 s against 4.3 s for MOPSO, and the runtime grows quickly with front size. Tier 1 runs once per region and weather window, not once per mission, so half a minute is acceptable, but it is still the main performance target for the next pass.

On S2 the HV values of 1.2065 and 1.2061 are too close to claim superiority without the 30 seeded MOPSO repetitions and a statistical test, which the next pass will run. The IGD result does not have this problem, since an IGD of zero cannot be beaten.

4.4.2 Effect of wind speed and direction

Table 4.5 shows the result of experiment E2.

TABLE 4.5: Effect of wind direction on the S2 front (25 km/h wind, 35 km/h gust)

Wind from	**Front**	**Labels**	**Min ε**	**Min ρ**	**Fragile pick** (d, ε, ρ)	**Distinct**
Calm	36	869	24.41	0.07	(26.78, 26.92, 0.11)	4
0° (N)	14	1019	31.01	3.24	(26.16, 32.82, 3.40)	2
45° (NE)	6	545	31.34	1.89	(26.16, 33.63, 1.89)	2
90° (E)	11	876	30.94	3.31	(26.16, 32.75, 3.45)	2
135° (SE)	14	1100	27.13	4.27	(26.16, 29.59, 4.42)	2
180° (S)	14	989	26.18	3.24	(26.16, 27.99, 3.40)	2
225° (SW)	6	482	24.58	1.89	(26.16, 26.87, 1.89)	2
270° (W)	11	801	26.22	3.31	(26.16, 28.03, 3.45)	2
315° (NW)	14	1084	27.21	4.27	(26.16, 29.67, 4.42)	2

The first observation is that direction alone matters a great deal. At the same speed, the cheapest route on the front needs 24.58 energy units with the wind from the south-west and 31.34 with it from the north-east, a difference of 27.5%. A planning algorithm using wind speed independent of its direction will cost both situations equally.

The two goals depend on different criteria. When the wind is rotated 180° (from 0° to 180° or from 45° to 225°), the lowest risk remains the same because of the crosswind, which depends only on the wind's axis. Minimum energy cannot be considered separately from the direction of the wind: the headwind cost is conditional on the direction of the movement of the drone with respect to the relevant axis. Both of them derive from Equation (3.4), which explains why headwind and crosswind are two separate cases.

The pattern of the front also depends on the state of the wind. In calm conditions, 36 routes are possible; under 8 wind directions, the number of available routes is 6–14, while the four presets give rise to just two unique routes. The reasoning here is that in wind, highly crosswind-unfavorable edges become not only expensive but risky, leading to a coincidence of costs and risks. The most sensitive case – fragile medical pick – always provides the safest path in wind, although the energy consumption of the route is wind-dependent (26.87–33.63). Repeating the run in 15° steps (Figure 4.2) gives a minimum energy of 24.6 to 31.6 units, a 28.5% swing, with 6 to 15 routes on the front.

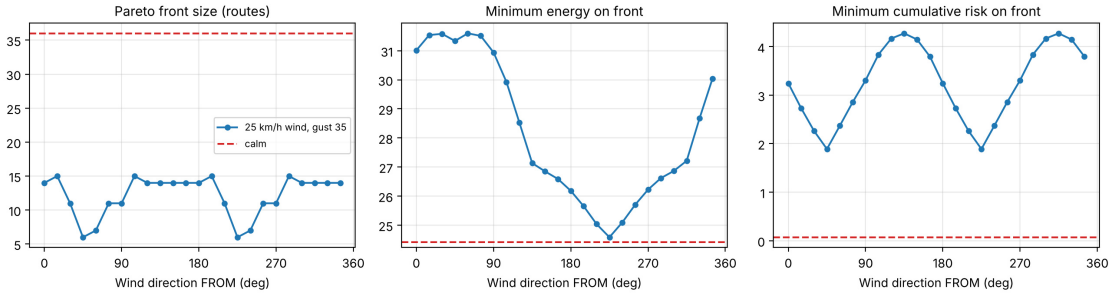


FIGURE 4.2: Front size, minimum energy and minimum risk on S2 as a 25 km/h wind turns through 360°; the dashed line is calm air

The collapse from four distinct picks to two may be the right answer for a front with fewer real trade-offs, or a side effect of min-max normalisation on a short front. The next pass will test both, and the answer may change Tier 2.

4.4.3 Directional edge costs

Table 4.6 gives the result of E3.

TABLE 4.6: Directional edge costs on a flat lattice, 25 km/h wind from 90°

Edge heading	0°	45°	90°	135°	180°	225°	270°	315°
ε/d	1.104	1.270	1.278	1.270	1.104	1.074	1.000	1.074
ρ_{edge}	0.225	0.170	0.038	0.170	0.225	0.170	0.038	0.170

Flying east, into the wind, costs 27.8% more energy per kilometre than flying west, which costs exactly the distance because a tailwind earns no credit. For north and south facing edges, the entire crosswind is present, and the risk score is six times that of east-west edges (0.225 compared to 0.038). The value of 0.038 is only that of the gust term for east-west edges. Still, the front on the flat lattice has only one route, as the straight diagonal is best for all goals. There is no trade-off resulting from the wind alone.

4.4.4 Contribution of each risk layer

E4 test results are tabulated in Table 4.7. The experiments used fixture layers, so the values show how the layers affect the planner.

TABLE 4.7: Ablation on the 6 km lattice (fixture layers, 30 km/h wind from 225°)

Variant	**Front**	**Time (s)**	**Min ε**	**Min ρ**	**Max ρ**	**Mean ρ_{edge}**
All layers	196	11.35	8.82	8.85	13.12	0.199
No wind	205	5.38	7.84	0.10	4.24	0.061
No terrain	56	1.36	8.81	8.81	13.08	0.198
No buildings	190	8.32	8.82	8.85	12.26	0.192

Wind matters most here: turning it off lowers the mean edge risk from 0.199 to 0.061, the safest path risk from 8.85 to 0.10, and the minimum energy by 11%.

Terrain, on the other hand, mostly controls how many routes there are. Without grades there are far fewer distinct trade-offs, 56 routes against 196, because altitude changes are what separate otherwise similar paths. Buildings raise the top limit of the risk range.

4.4.5 Tier 2 selection

Table 4.8 gives the Tier 2 picks of E5 on the S2 and S3 fronts. The ranks are the selected route's position on each objective within the front, where 1 is best.

TABLE 4.8: Tier 2 picks on the S2 (16 routes) and S3 (115 routes) fronts

Front	Preset	d	ε	ρ	Rk d	Rk ε	Rk ρ	Point type	ms
S2	Fragile medical	19.45	20.94	0.69	7	5	10	non-extreme	0.030
	Emergency medicine	19.15	20.56	0.81	3	1	14	min-energy	0.025
	Disaster relief	19.15	20.56	0.81	3	1	14	min-energy	0.022
	Mixed cargo	19.22	20.66	0.75	4	2	13	non-extreme	0.023
S3	Fragile medical	41.02	45.86	1.92	91	81	25	non-extreme	0.144
	Emergency medicine	37.11	44.71	6.56	22	26	94	non-extreme	0.140
	Disaster relief	37.52	44.27	5.63	28	1	88	min-energy	0.136
	Mixed cargo	39.90	45.06	2.74	84	59	32	non-extreme	0.136

On the 115-route S3 front, the four presets take four separate routes, three of which are represented by non-extreme points. Disaster relief uses the minimum energy path, consistent with its dominant weight.

The emergency medicine preset deserves a special consideration. It is merely the 22nd shortest route, yet it is just 1.9% longer than the shortest route in the front. While the distance is the key factor in the setting of its weights, it is not the only one; therefore, the utility function sacrifices some distance for smaller risk. A single objective planner cannot do that kind of tradeoff, and a fixed weights planner would make the same tradeoff for each payload.

The fragile medical and emergency routes show the gap between Tier 2 options. The fragile route is 10.5% longer than the emergency one (41.02 versus 37.11), and it has 71% less risk (1.92 versus 6.56). It ranks 25th out of 115 on risk instead of 1st, because both distance and energy coefficients are still positive and the more cautious paths are much longer. Mixed cargo is in an intermediate position with respect to all criteria, as expected from its balanced weights. Selections on S3 took about 0.14 ms, against a target of one second.

On the smaller S2 front, emergency medicine and disaster relief take the same minimal energy path, as in the wind example: small fronts leave Tier 2 fewer paths.

Figure 4.3 shows the four presets on a 2 km lattice (11-route front) with OpenStreetMap roads drawn underneath for reference. The fragile medical and mixed cargo presets take a slightly longer path (2.6 km) with a cumulative risk of 1.57, while emergency medicine and disaster relief take 2.5 km paths with risks of 1.88 and 1.89.

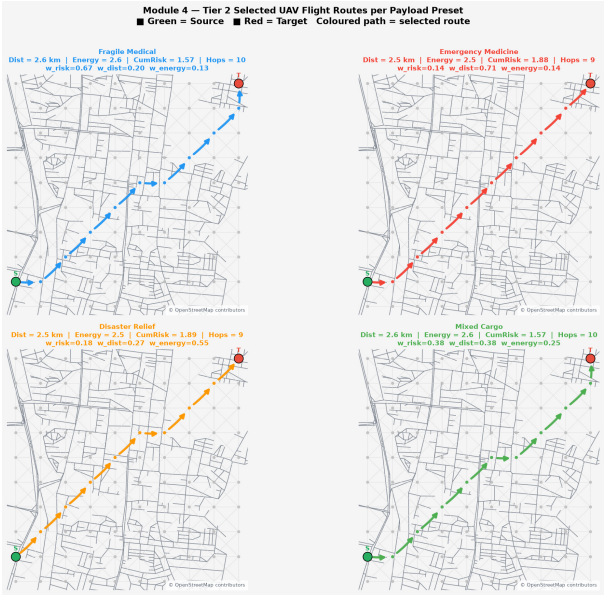


FIGURE 4.3: Tier 2 routes for the four presets on the 2 km lattice

The first monotonicity check reported 660 violations on the smallest benchmark, but it had compared selections across scenarios; grouped by scenario, the count is zero.

4.4.6 Simulation results

In E6 the SITL mission runner flew planner missions against ArduCopter SITL. Mission upload, read-back, the switch to AUTO, the landing and the disarm all succeeded. The cross-track root-mean-square error between the flown and planned tracks was 0.11–0.13 m, and the landing offset from the destination pad was 0.01–0.02 m. These numbers show that the mission chain from Tier 2 to the autopilot is correct, including the altitude frame, which is easy to get wrong over uneven terrain.

Figure 4.4 shows a Tier 2 mission flown in AUTO inside the Gazebo city world, with the MAVProxy console on the left and the mission in QGroundControl on the right. The Gazebo world has no wind yet, so logged comparisons across wind cases are planned for the next pass.

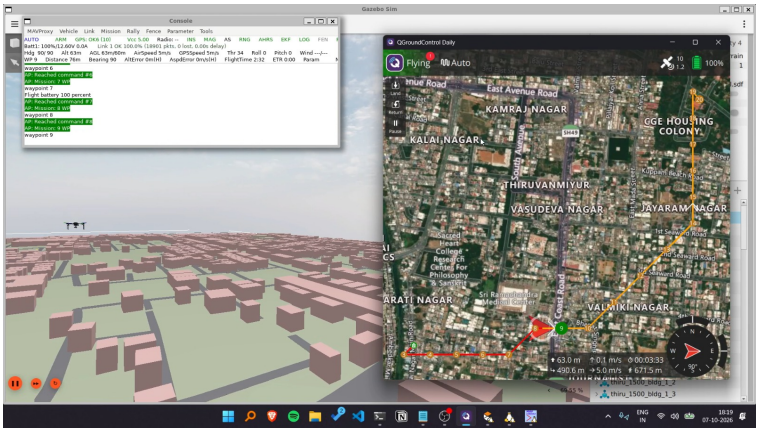


FIGURE 4.4: Gazebo Harmonic and ArduPilot SITL flying an uploaded Tier 2 mission in AUTO: MAVProxy console (left), QGroundControl mission view (right)

4.4.7 Comparison with existing work

Table 4.9 compares the system with the planners reviewed in Chapter 2.

TABLE 4.9: Comparison with the reviewed planners

Property	**Chokkalingam and Ganesan [4]**	**Ren et al. [18]**	**Deng et al. [7]**	**Tsoutsouras et al. [21]**	**This work**
Front construction	POPSO, stochastic	CDNSGA-II, stochastic	MS-INSGA2, stochastic	None; one trajectory	NAMOA*, exact and deterministic
Objectives	Length, energy, safety, obstacle penalty, weather factor	Distance, safety	Length, collision risk, smoothness	Deadline misses under energy and mass limits	Distance, energy, risk (11 components)
Final route selection	Weighted sum, weights set per mission	Crowding distance (k nearest)	Not discussed	Not applicable	Utility from payload tags, argmax over the full front
Payload in selection	No	No	No	Mass in the power model only	Fragility, weight class, urgency, priority
Wind	Speed-only factor, uniform	Not modelled	Future work	Not modelled	Headwind and crosswind per edge, gust, shear, weather gate
Front reuse	Re-run per mission	Re-run per mission	Re-run per task	Not applicable	Stored front with drift-based refresh

The comparison is qualitative, as their datasets and code are not public; the next pass scores their selection rules on our fronts (Section 3.6).

CHAPTER 5

SOCIAL IMPACT AND SUSTAINABILITY

5.1 SOCIAL IMPACT

Drone delivery helps most the people who live far from a hospital or are cut off by an emergency. Rural healthcare logistics considerations would be in cases where there was a need for blood or an antivenom in a clinic, a town surrounded by floods, or regular delivery of samples from a hospital to a laboratory in one day. Similar services already exist, such as Zipline drones in Rwanda [9] and hospital samples flights in Switzerland [2]. The suggested method adds a way for each flight to adapt to the particular cargo, where urgent antivenom would fly through the shortest path while fragile cargo would use another path. This advantage is important since there might be a situation where no specialists in planning will be available. The launch of a drone from a district hospital or a relief camp will be conducted either by a nurse or volunteers instead of a specialist in planning. This person needs only to provide four simple answers about the cargo characteristics (frangible, weight, urgency, and importance), after which all other decisions will be made automatically.

Underneath, there are people who do not want the drone to fly above their heads. Every edge cost considers aspects such as population, restricted areas, water distance, and building distance. All nodes that exceed the statutory limit are disregarded, and adverse weather conditions make it impossible for flights if there

are storms, rainfall, poor visibility, and high winds. There is no way that the planner will go through a crowded market just to save a few seconds.

All decisions made at Tier 2 level are saved along with the corresponding tags, weights, route, and any HOLD reason. The fragility tag is a precaution and has not been shown to protect anything [14, 19, 20]. Until proper risk weights are set, the derived risk values will act as relative rankings of the routes rather than probabilities of harm. Phase II outdoor flights will follow the Drone Rules, 2021 [13].

5.2 SUSTAINABILITY

For the short-distance flights of light payloads, an electric-powered drone may be substituted for van or motorcycle transport without exhaust emission during travel. The planner helps in two major ways. Firstly, its energy goal considers the wind direction, which alone changed the energy of the cheapest route by 27.5%, implying that headwinds may be avoided. Secondly, the payload weight label gives more weight to energy criteria for heavy payloads. Flights whose battery energy is exhausted to less than the 20% reserve level are excluded, thereby assuring both the success of the flight and battery longevity.

The computational demand is low as well. Tier 1 runs on an ordinary laptop CPU with no GPU or cloud service, once per region and weather window. Tier 2 takes a fraction of a millisecond and can run on a Raspberry Pi on board. Everything the system depends on is open data or open-source software, so there are no licence fees.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project set out to let the payload decide the route. Existing multi-objective UAV planners build good Pareto fronts and then choose from them with a rule that knows nothing about the cargo, or with weights an expert re-tunes for every mission. We separated the two problems. Tier 1 builds an exact NAMOA* front over a three-dimensional, wind-aware airspace lattice once per region and weather window, and stores it until it goes stale. Tier 2 turns four payload tags into objective weights with an explicit formula, scores every route on the front and returns the best one, or a HOLD when the weather or the battery rules the mission out. The chosen route has been flown in SITL and in a Gazebo city world.

The preliminary results have been quite encouraging, but there are still some important observations to make. First, the Tier 1 front was exact for each of the benchmarks, while MOPSO managed to discover 38 routes out of 115 on the biggest one. On that front the four reference payloads received four different routes, three of them non-extreme, in about 0.14 ms each. Secondly, when only the wind direction was varied at constant speed, the energy consumed on the route that had the lowest cost changed by 27.5%. The results of the SITL simulations showed that the drone stayed within 0.13 m RMS of the planned trajectory. However, there were a number of flaws found as well. In a steady wind the front shrank and the presets fell onto two routes; MOPSO ran at reduced

settings; the ablation used fixture data; and the risk weights and battery constants are still placeholders.

6.2 FUTURE WORK

The following experiment conducts the full benchmark (four graphs, nine wind cases, MOPSO set to use 100 particles, 200 iterations, and 30 runs) with stats for each run and significance test. Density tags and monotonicity classification need fixing, and our goal is to find out the reason for Pareto collapse in constant wind case by testing other normalization and weight multipliers. Ablation and Tier 2 experiments will be performed again but using real map data, and then Tier 2 will be tested against crowding distance, knee point, shortest route, and fixed weights for all 225 tag combinations. There are two flaws in the dashboard that are not fixed yet, and Tier 1 needs better performance in large graphs. Simulation involves putting the Gazebo world through wind, tuning the battery model according to the class of load it carries, and running the pre-defined setup for all different wind conditions with full flight logs recorded. For Phase II, the system switches to using an F450 quadcopter with the Pixhawk and a companion computer called Raspberry Pi 5 and a servo for releasing the payload. The Tier 2 system runs on the drone, and outdoor testing will involve checking the extent to which it stays on course and the time between entering tags and starting the mission. Following this, the two-tier system can be further expanded to include multiple drones having a common front, performing multi-drop delivery missions, or in-flight re-planning of routes based on the changing weather.

REFERENCES

1. ArduPilot Development Team. (2026). *ArduPilot software in the loop (SITL) simulator*. https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html
2. Avionics International. (2019, August 8). *Drone delivery crash in Switzerland raises safety concerns as UPS forms subsidiary*. https://www.aviationtoday.com/2019/08/08/drone-delivery-crash-in-switzerland-raises-safety-concerns/
3. Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, *65*, 126–139. https://doi.org/10.1016/j.compenvurbsys.2017.05.004
4. Chokkalingam, K., & Ganesan, R. (2026). Dynamic parameter adaptation in Pareto-optimal particle swarm optimization for efficient UAV path planning. *RAIRO – Operations Research*, *60*(1), 281–305. https://doi.org/10.1051/ro/2025159
5. Coello Coello, C. A., Toscano Pulido, G., & Salazar Lechuga, M. (2004). Handling multiple objectives with particle swarm optimization. *IEEE Transactions on Evolutionary Computation*, *8*(3), 256–279. https://doi.org/10.1109/TEVC.2004.826067
6. Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, *6*(2), 182–197. https://doi.org/10.1109/4235.996017
7. Deng, J., Zhang, H., Feng, D., Yu, Z., & Liu, Z. (2026). Multi-objective UAV trajectory planning algorithm based on the hybrid MS-INSGA2 strategy. *Journal of King Saud University – Computer and Information Sciences*, *38*, Article 562. https://doi.org/10.1007/s44443-026-00930-0
8. Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)* (pp. 11–15). https://doi.org/10.25080/TCWV9851
9. IEEE Spectrum. (2019). *In the air with Zipline's medical delivery drones*. https://spectrum.ieee.org/in-the-air-with-ziplines-medical-delivery-drones
10. Joint Authorities for Rulemaking of Unmanned Systems. (2019). *JARUS guidelines on specific operations risk assessment (SORA)* (JAR-DEL-WG6-D.04, Edition 2.0).
11. Mandow, L., & Pérez de la Cruz, J. L. (2010). Multiobjective A* search with consistent heuristics. *Journal of the ACM*, *57*(5), Article 27. https://doi.org/10.1145/1754399.1754400
12. Meier, L., et al. (2013). *MAVLink: Micro air vehicle communication protocol*. https://mavlink.io
13. Ministry of Civil Aviation, Government of India. (2021). *The Drone Rules, 2021* (G.S.R. 589(E)). Gazette of India.

14. Oakey, A., Waters, T., Zhu, W., Royall, P. G., Cherrett, T., Courtney, P., Majoe, D., & Jelev, N. (2021). Quantifying the effects of vibration on medicines in transit caused by fixed-wing and multi-copter drones. *Drones*, *5*(1), 22. https://doi.org/10.3390/drones5010022
15. Open-Meteo. (2026). *Free weather API and elevation API*. https://open-meteo.com
16. Open Robotics. (2023). *Gazebo Harmonic*. https://gazebosim.org
17. Peterson, E. W., & Hennessey, J. P. (1978). On the use of power laws for estimates of wind power potential. *Journal of Applied Meteorology*, *17*(3), 390–394. https://doi.org/10.1175/1520-0450(1978)017<0390:OTUOPL>2.0.CO;2
18. Ren, Q., Yao, Y., Yang, G., & Zhou, X. (2019). Multi-objective path planning for UAV in the urban environment based on CDNSGA-II. In *Proceedings of the IEEE International Conference on Service-Oriented System Engineering (SOSE)*. https://doi.org/10.1109/SOSE.2019.00059
19. Stierlin, N., Loertscher, F., Renz, H., Risch, L., & Risch, M. (2024). Preanalytic integrity of blood samples in uncrewed aerial vehicle (UAV) medical transport: A comparative study. *Drones*, *8*(9), 517. https://doi.org/10.3390/drones8090517
20. Theobald, K., Zhu, W., Waters, T., Cherrett, T., Oakey, A., & Royall, P. G. (2023). Stability of medicines transported by cargo drones: Investigating the effects of vibration from multi-stage flight. *Drones*, *7*(11), 658. https://doi.org/10.3390/drones7110658
21. Tsoutsouras, V., Story, J., & Stanley-Marbell, P. (2020). *Payload-mass-aware trajectory planning on multi-user autonomous unmanned aerial vehicles* (arXiv:2001.02531). arXiv. https://doi.org/10.48550/arXiv.2001.02531
22. Zitzler, E., & Thiele, L. (1999). Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, *3*(4), 257–271. https://doi.org/10.1109/4235.797969

APPENDIX A

CONFIGURATION PARAMETERS

Adjustable parameters are all located in the `config.py` file. Table A.1 lists the main parameters of the airspace graph and cost model, and Table A.2 those of the front store and Tier 2. Values marked provisional are placeholders that will be replaced by calibrated values.

TABLE A.1: Main airspace graph and cost model parameters

Parameter	**Value**	**Meaning**
`BOX_M`, `LATTICE_SPACING_M`	6000 m, 200 m	Lattice extent and spacing
`H_CRUISE_AGL`, `CLEARANCE`	60 m, 15 m	Cruise height, margin above roofs
`H_MAX_AGL`	120 m	Legal ceiling above ground
`V_CRUISE_KMH`	36 km/h	Airspeed used in the wind terms
`BETA_HEAD`, `BETA_CROSS`	0.40, 0.15	Headwind and crosswind energy coefficients
`V_MAX_SAFE_KMH`, `K_GUST`	40 km/h, 0.5	Wind risk saturation speed, gust weight
`ALPHA_SHEAR`	0.25	Power-law wind shear exponent
`EDGE_SAMPLE_STEP_M`	25 m	Sampling step along each edge
`NFZ_SOFT_LAMBDA_M`	150 m	Decay length of the no-fly buffer
`NO_GO_GUST_KMH`	45 km/h	Gust limit of the weather gate (provisional)
`RISK_WEIGHTS`	Table 3.1	Weights of the eleven risk components (uncalibrated)

TABLE A.2: Front store and Tier 2 parameters

Parameter	Value	Meaning
Front validity window	3 h	Front expires after this time
Wind speed drift	10 km/h	Rebuild when wind speed moves more than this
Wind direction drift	45° (wind > 10 km/h)	Rebuild when wind turns more than this
Gust drift	15 km/h	Rebuild when gust moves more than this
Tier 1 expansion cap	200,000 (S1/S2/S3: 80k, 180k, 260k)	Guards against runaway search
Payload masses m_W	0.25, 0.5, 1.0 kg	Nominal mass of weight classes 1, 2, 3
m_{uav}, battery, reserve	1.5 kg, 74 Wh, 20%	Airframe mass and battery (provisional)
k_{cal}	6 Wh per unit	Energy proxy to watt-hours (provisional)
Priority threshold	$P_{\text{lvl}} \geq 4$	Dominant weight amplified by 10%

APPENDIX B

AUTOMATED TEST SUITES

Both code bases have offline unit-test suites that need no network and no API key. The planner suite has about 220 tests and the simulation suite 45. Table B.1 lists what they check.

TABLE B.1: Test suites and what they verify

Test file	What it checks
`test_wind.py`	Headwind sign (wind from 90° is a headwind for an east-bound edge), crosswind, power-law shear, hourly row selection
`test_lattice.py`	Lattice size, 8-neighbour edges, east-bound edges cost more than west-bound edges under an east wind

Test file	**What it checks**
`test_terrain.py`	DEM provider chain and cache, node altitude rule, every node within 120 m above ground
`test_risk.py`	Risk components in $[0, 1]$, restricted-zone node and edge removal, soft buffer, weather gate
`test_tier1_front.py`	NAMOA* front equals a brute-force front over all simple paths on a small graph; no route dominates another; every path valid
`test_store.py`	Reuse, expiry, graph-hash change, each drift threshold, identity check
`test_tier2_payload.py`, `test_tier2_hardening.py`	Weight formula, amplification, non-extreme pick for mixed cargo on a stored lattice front, monotonicity, validation errors, latency under 1 s, battery HOLD
`test_evaluation_metrics.py`	HV, IGD and spread on hand-computed cases; alignment score; competing pickers
`test_mission_export.py`	QGC WPL 110 round trip, collinear merge, frame 3 altitude relative to home
`test_ablation.py`, `test_overpass.py`, `test_dotenv.py`, `test_dashboard_helpers.py`	Ablation smoke run, Overpass retry and parsing, key loading, dashboard helpers
`Simulation/tests/*`	Bundle loading and frames, clearance checker, mission export (take-off, route, land), world builder geometry

APPENDIX C

CODE ORGANISATION

The planner is written in Python as flat scripts, one file per module, plus a configuration file, an environment-data file and entry-point scripts. The simulation lives in a separate folder and imports the planner modules through the Python path. Every tunable constant (region, flight heights, wind coefficients, risk weights, battery and mass values, drift thresholds, Overpass endpoints) lives in one file, `config.py`, so an experiment changes a value in one place. API keys are read from a `.env` file by the entry-point scripts only, so the test suites never need a key or a network connection. Table C.1 maps the source files to the modules.

TABLE C.1: Source files and the modules they implement

File	Module	Role
`config.py`	all	Every tunable constant
`env_data.py`	1	Weather, DEM providers, OSM layers, spatial indexes, key loader
`overpass_client.py`	1	Multi-endpoint Overpass fetch with retry
`module1_graph_builder.py`	1	Synthetic and lattice builders, cost pipeline
`module2_tier1_moastar.py`	2	NAMOA* engine, dominance helpers, thinning
`pareto_store.py`	2	Front store, graph hash, drift, region runs
`module3_baselines.py`	3	A* variants, weighted-sum A*, MOPSO
`module4_tier2_payload.py`	4	Weight bridge, selector, feasibility, export
`module5_evaluation.py`	5	HV, IGD, spread, Tier 2 evidence, sweep
`run_benchmarks.py`, `run_ablation.py`	5	Benchmark and ablation runners
`mission_export.py`, `dashboard.py`	6	QGC WPL 110 export, operator interface
`run_demo_syn.py`, `run_demo_osm.py`	1–4	End-to-end demonstrations with figures
`Simulation/*.py`	7	Bundle, clearance, world, mission, runner, logger, evaluator